VECTOR

FBL LibMem

Technical Reference

Flash Bootloader - Memory Handling Library

Version 1.0.0

Authors	Sebastian Loos
Status	Released

Document Information

History

Author	Date	Version	Remarks
Sebastian Loos	2019-01-31	1.00.00	Initial Version

Reference Documents

No.	Source	Title	Version
[1]		AN-ISC-8-1234 Multiprocessor Bootloader	-



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2019 Vector Informatik GmbH

Contents

1 Component History 10

2 Introduction 11
2.1 Architecture Overview 11
2.2 Definitions 12

3 Functional Description 14
3.1 Features 15
3.1.1 Gap Fill 15
3.1.2 Pipelined Programming 16
3.1.2.1 Suspend Programming 17
3.1.3 Pipelined Verification 17
3.1.4 Data Processing 18
3.1.5 Stream Output 19
3.1.6 Passthrough 20
3.1.7 Resumable Programming 22
3.1.8 Multiple Input Sources 25
3.2 Verification Variants 25
3.3 Pipelined Operations 26
3.4 States 27
3.5 Main Functions 28
3.6 Error Handling 29
3.6.1 Development Error Reporting 29

4 Integration 30
4.1 Scope of Delivery 30
4.1.1 Static Files 30
4.1.2 Other Files 30

5 API Description 31
5.1 Type Definitions 31
5.1.1 tFblMemVerifyRoutineInput 31
5.1.2 tFblMemVerifyRoutineOutput 31
5.1.3 tFblMemBlockInfo 32
5.1.4 tFblMemVerifyData 33
5.1.5 tFblMemBlockVerifyData 33
5.1.6 tFblMemSegmentInfo 33
5.1.7 Logical and Physical Addresses 33
5.1.8 Data Format Identifier 34

5.2 General Services provided by LibMem ... 34
5.2.1 InitPowerOn ... 34
5.2.2 InitPowerOnExt ... 34
5.2.3 Init ... 35
5.2.4 Deinit ... 35
5.2.5 GetActiveBuffer ... 36
5.2.6 BlockEraseIndication ... 36
5.2.7 BlockStartIndication ... 37
5.2.8 BlockEndIndication ... 37
5.2.9 BlockVerify ... 38
5.2.10 SegmentStartIndication ... 38
5.2.11 SegmentEndIndication ... 39
5.2.12 DataIndication ... 40
5.2.13 Task ... 40
5.2.14 FlushInputData ... 41
5.3 Resumable Programming ... 41
5.3.1 InvalidateSnapshot ... 41
5.3.2 CreateSnapshot ... 42
5.3.3 RestoreSnapshot ... 43
5.3.4 GetSnapshotInfo ... 43
5.4 Low level memory operations ... 44
5.4.1 FblMemEraseRegion ... 44
5.4.2 FblMemProgramBuffer ... 45
5.5 Helper functions ... 45
5.5.1 FblMemSetInteger ... 45
5.5.2 FblMemGetInteger ... 46
5.6 Multi Source ... 46
5.6.1 FblMemLockInputSource ... 46
5.7 Error Handling ... 47
5.7.1 FblMemRemapStatus ... 47
5.8 Services used by LibMem ... 47
5.9 Configurable Interfaces ... 48
5.9.1 Notifications ... 48
5.9.1.1 RxNotification ... 48
5.9.1.2 FblMemResumeIndication ... 48
5.9.2 Callout Functions ... 49
5.9.3 General Callouts ... 49
5.9.3.1 __ApplFblMemCopyBuffer ... 49
5.9.3.2 __ApplFblMemEnterCriticalSection ... 50
5.9.3.3 __ApplFblMemLeaveCriticalSection ... 50
5.9.3.4 __ApplFblMemReportProgress ... 50

5.9.4 Memory access Callouts 51
5.9.4.1 __ApplFblMemDriverReady 51
5.9.4.2 __ApplFblMemConditionCheck 51
5.9.4.3 __ApplFblMemPreWrite 52
5.9.4.4 __ApplFblMemPostWrite 52
5.9.4.5 __ApplFblMemPreErase 53
5.9.4.6 __ApplFblMemPostErase 53
5.9.4.7 __ApplFblMemPostSegmentEnd 54
5.9.5 Watchdog and Response Pending Callouts 54
5.9.5.1 __ApplFblMemWdTrigger 54
5.9.5.2 __ApplFblMemAdaptiveRcrRp 55
5.9.5.3 __ApplFblMemForcedRcrRp 55
5.9.5.4 __ApplFblMemIsRcrRpActive 55
5.9.6 Resumable Programming Callouts 56
5.9.6.1 ApplFblMemCheckpointIndication 56
5.9.7 Suspend Programming Callouts 56
5.9.7.1 ApplFblMemDriverSuspendWrite 56
5.9.7.2 ApplFblMemDriverResumeWrite 57
5.9.7.3 ApplFblMemDriverGetWriteRemainder 57
5.9.8 Passthrough Callouts 58
5.9.8.1 __ApplFblMemIsPassThroughActive 58
5.9.8.2 __ApplFblMemDataIndication 58
5.9.8.3 __ApplFblMemBlockEraseIndication 59
5.9.8.4 __ApplFblMemBlockStartIndication 59
5.9.8.5 __ApplFblMemBlockEndIndication 60
5.9.8.6 __ApplFblMemSegmentStartIndication 60
5.9.8.7 __ApplFblMemSegmentEndIndication 61
5.9.8.8 __ApplFblMemBlockVerify 62
5.9.9 Verification Callouts 62
5.9.9.1 __ApplFblMemVerifyInputIsAddress… 63
5.9.9.2 __ApplFblMemVerifyProcessedIsAddress… 63
5.9.9.3 __ApplFblMemVerifyPipelinedIsAddress… 64
5.9.10 Stream Output Callouts 64
5.9.10.1 __ApplFblMemIsStreamOutputRequired 64
5.9.10.2 ApplFblInitStreamOutput 65
5.9.10.3 ApplFblStreamOutput 65
5.9.10.4 ApplFblFinalizeStreamOutput 66
5.9.10.5 ApplFblDeinitStreamOutput 67
5.9.11 Data Processing Callouts 68
5.9.11.1 __ApplFblMemIsDataProcessingRequired 68
5.9.11.2 ApplFblInitDataProcessing 68

5.9.11.3 ApplFblDataProcessing 69
5.9.11.4 ApplFblDeinitDataProcessing 69
5.9.12 Pipelining Callouts 70
5.9.12.1 __ApplFblMemIsPipelinedProgramming 70

6 Configuration 71
6.1 Configuration manually 71

7 Glossary and Abbreviations 72
7.1 Glossary 72
7.2 Abbreviations 72

8 Contact 73

Illustrations

Figure 2-1 FBL Architecture Overview 11
Figure 2-2 Interfaces to adjacent modules of the LibMem 12
Figure 2-3 Blocks, Segments 13
Figure 3-1 Data Flow 14
Figure 3-2 Gap Fill 15
Figure 3-3 Pipelined Programming 16
Figure 3-4 Pipelined Programming with Data Processing 16
Figure 3-5 Pipelined Verification 17
Figure 3-6 Pipelined Verification Program Flow 18
Figure 3-7 States of data processing 19
Figure 3-8 Stream Output Program Flow 20
Figure 3-9 The Passthrough interface re-routes request from the upper layer (Diag) directly to a Sub-µC 21
Figure 3-10 If Pipelined Programming is enabled, one DataIndication from the upper layer (diagnostics) is split into multiple pipelined Passthrough-DataIndications, each with a reduced number of data 21
Figure 3-11 Passthrough Interface 22
Figure 3-12 Other diagnostic services (Non-LibMem services) need to be routed separately. 22
Figure 3-13 Resumable Programming 23
Figure 3-14 Resumable Programming Flow 24
Figure 3-15 Multiple Input Sources (principle) 25
Figure 3-16 Reduced download speed on Pipelined Verification because of large VERIFY_SEGMENTATION size 27
Figure 3-17 State flow of the LibMem 28

Tables

Table 1-1 Component history 10
Table 4-1 Static files 30
Table 4-2 Other Files 30
Table 5-1 Type definitions 31
Table 5-2 tFblMemVerifyRoutineInput 31
Table 5-3 tFblMemVerifyRoutineOutput 32
Table 5-4 tFblMemBlockInfo 32
Table 5-5 tFblMemVerifyData 33
Table 5-6 tFblMemBlockVerifyData 33
Table 5-7 tFblMemSegmentInfo 33
Table 5-8 InitPowerOn 34
Table 5-9 InitPowerOnExt 35
Table 5-10 Init 35
Table 5-11 Deinit 36
Table 5-12 GetActiveBuffer 36
Table 5-13 BlockEraseIndication 37
Table 5-14 BlockStartIndication 37
Table 5-15 BlockEndIndication 38
Table 5-16 BlockVerify 38
Table 5-17 SegmentStartIndication 39
Table 5-18 SegmentEndIndication 40
Table 5-19 DataIndication 40
Table 5-20 Task 41

Table 5-21 FlushInputData 41
Table 5-22 InvalidateSnapshot 42
Table 5-23 CreateSnapshot 43
Table 5-24 RestoreSnapshot 43
Table 5-25 GetSnapshotInfo 44
Table 5-26 FblMemEraseRegion 45
Table 5-27 FblMemProgramBuffer 45
Table 5-28 FblMemSetInteger 46
Table 5-29 FblMemGetInteger 46
Table 5-30 FblMemLockInputSource 47
Table 5-31 FblMemRemapStatus 47
Table 5-32 Services used by the LibMem 48
Table 5-33 RxNotification 48
Table 5-34 FblMemResumeIndication 49
Table 5-35 __ApplFblMemCopyBuffer 49
Table 5-36 __ApplFblMemEnterCriticalSection 50
Table 5-37 __ApplFblMemLeaveCriticalSection 50
Table 5-38 __ApplFblMemReportProgress 51
Table 5-39 __ApplFblMemDriverReady() 51
Table 5-40 __ApplFblMemConditionCheck 52
Table 5-41 __ApplFblMemPreWrite() 52
Table 5-42 __ApplFblMemPostWrite 53
Table 5-43 __ApplFblMemPreErase 53
Table 5-44 __ApplFblMemPostErase 53
Table 5-45 __ApplFblMemPostSegmentEnd 54
Table 5-46 __ApplFblMemWdTrigger 54
Table 5-47 __ApplFblMemAdaptiveRcrRp 55
Table 5-48 __ApplFblMemForcedRcrRp 55
Table 5-49 __ApplFblMemIsRcrRpActive 56
Table 5-50 ApplFblMemCheckpointIndication 56
Table 5-51 ApplFblMemDriverSuspendWrite 57
Table 5-52 ApplFblMemDriverResumeWrite 57
Table 5-53 ApplFblMemDriverGetWriteRemainder 58
Table 5-54 __ApplFblMemIsPassThroughActive 58
Table 5-55 __ApplFblMemDataIndication 59
Table 5-56 __ApplFblMemBlockEraseIndication 59
Table 5-57 __ApplFblMemBlockStartIndication 60
Table 5-58 __ApplFblMemBlockEndIndication 60
Table 5-59 __ApplFblMemSegmentStartIndication 61
Table 5-60 __ApplFblMemSegmentEndIndication 62
Table 5-61 __ApplFblMemBlockVerify 62
Table 5-62 __ApplFblMemVerifyInputIsAddressAndLengthIncluded 63
Table 5-63 __ApplFblMemVerifyProcessedIsAddressAndLengthIncluded 63
Table 5-64 __ApplFblMemVerifyPipelinedIsAddressAndLengthIncluded 64
Table 5-65 ApplFblMemIsStreamOutputRequired 64
Table 5-66 ApplFblInitStreamOutput 65
Table 5-67 ApplFblStreamOutput 66
Table 5-68 ApplFblFinalizeStreamOutput 67
Table 5-69 ApplFblFinalizeStreamOutput 68
Table 5-70 __ApplFblMemIsDataProcessingRequired 68
Table 5-71 ApplFblInitDataProcessing 69
Table 5-72 ApplFblDataProcessing 69
Table 5-73 ApplFblDeinitDataProcessing 70
Table 5-74 __ApplFblMemIsPipelinedProgrammingDisabled 70

Table 7-1 Glossary 72
Table 7-2 Abbreviations 72

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.00	Initial release.
1.04.00	Support of Passthrough interface, Block Erase, Output Verification and error code remapping.
1.06.00	Support of Pipelined Verification.
2.00.00	Raised major version to match API version.
3.00.00	Support of Stream Output, Resumable Programming, Processed Verification and selective Pipelined Programming.
3.01.00	Support of Gap Fill.
4.00.00	Support combination of Input and Processed Verification.

Table 1-1 Component history

2 Introduction

The full name of the LibMem component is FblLib_Mem. Hence, it is usually referenced as LibMem – therefore this naming is used here. The LibMem abstracts the memory access for an upper layer, e.g. the diagnosis. It offers a variety of configuration options to address the needs of different OEMs requirements and data processing features.

2.1 Architecture Overview

The following figure shows where the LibMem is in the FBL architecture.

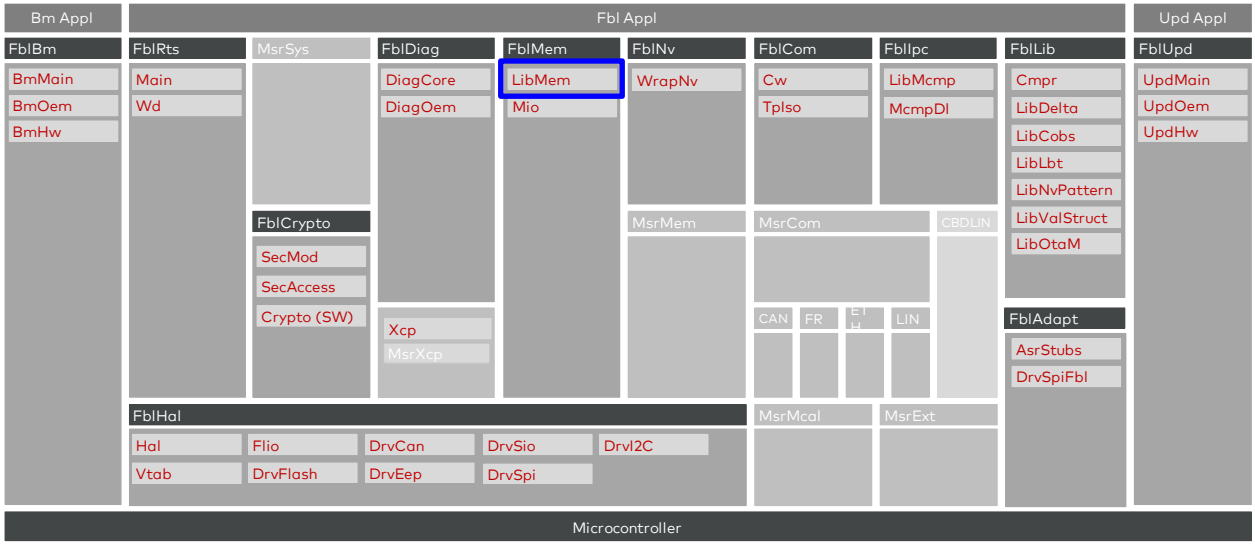


Figure 2-1 FBL Architecture Overview

The next figure shows the interfaces to adjacent modules of the LibMem. These interfaces are described in paragraph 5.

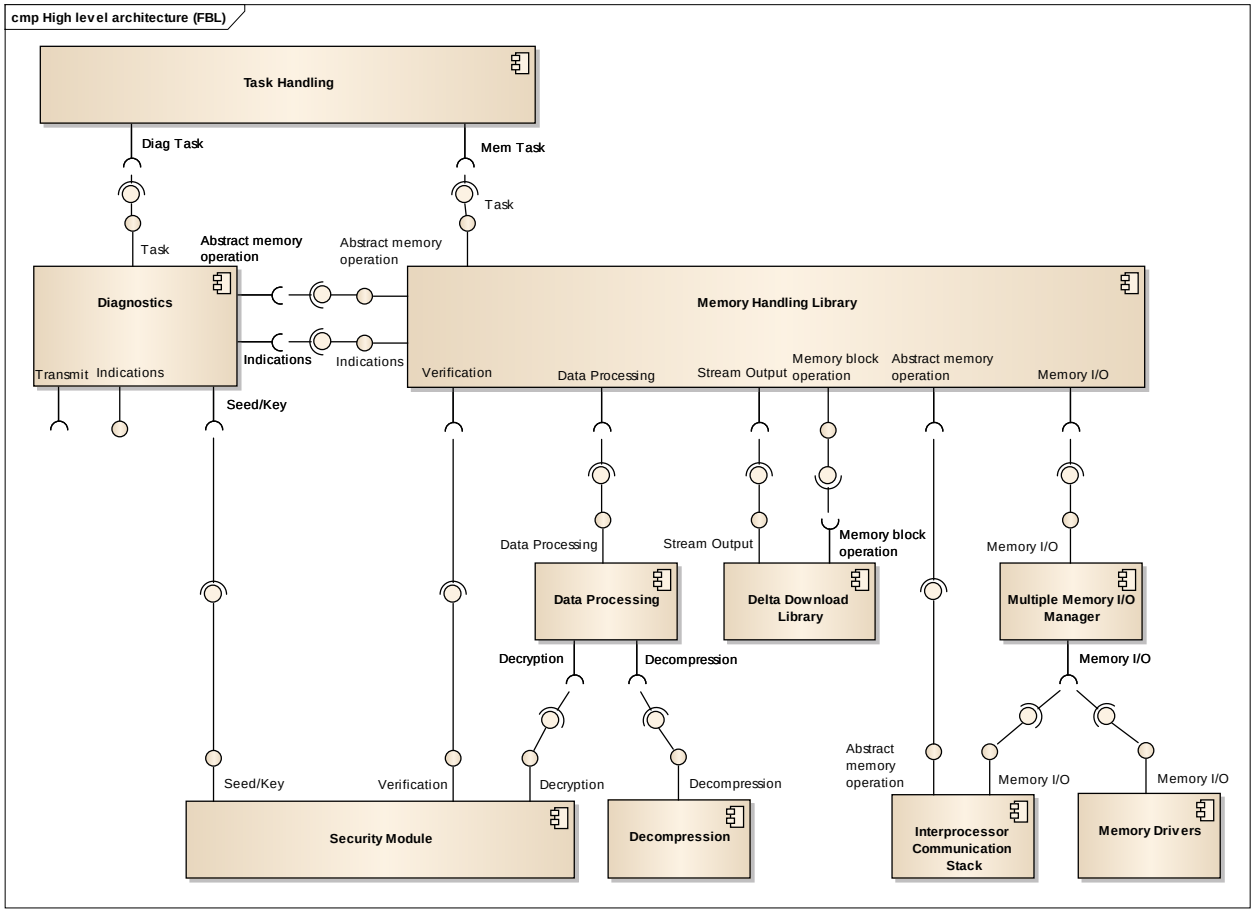


Figure 2-2 Interfaces to adjacent modules of the LibMem

2.2 Definitions

A **block** is a memory region which is

- aligned to the erase size and
- has a length aligned to the erase size.
- In the Bootloader context a block is referenced as a *Logical Block*.
- A block consists of contiguous memory as defined by the Flash Block Table (FBT). If the FBT is non-contiguous (as shown in Figure 2-3), The block can spread over the gaps.

A **segment** is a contiguous part of memory inside a block. Its start address needs to be aligned to the Flash Segment Size (see below). The end address can be unaligned.

The **Flash Segment Size** is the smallest size of writable data.

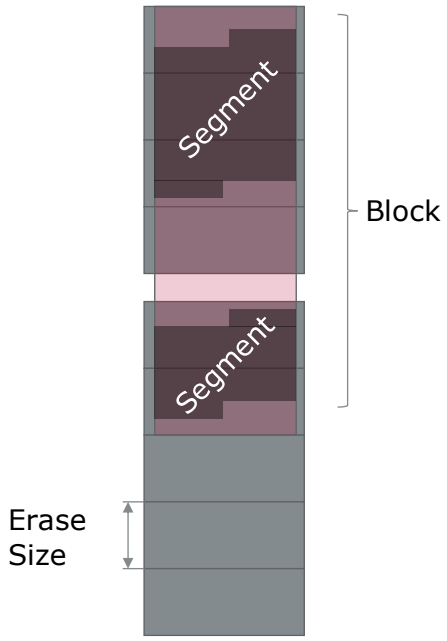


Figure 2-3 Blocks, Segments

3 Functional Description

The LibMem abstracts the access of (flash) memory in the context of a FBL download. It is highly optimized to allow a minimum usage of memory and allows various modes of handling download data for maximum performance.

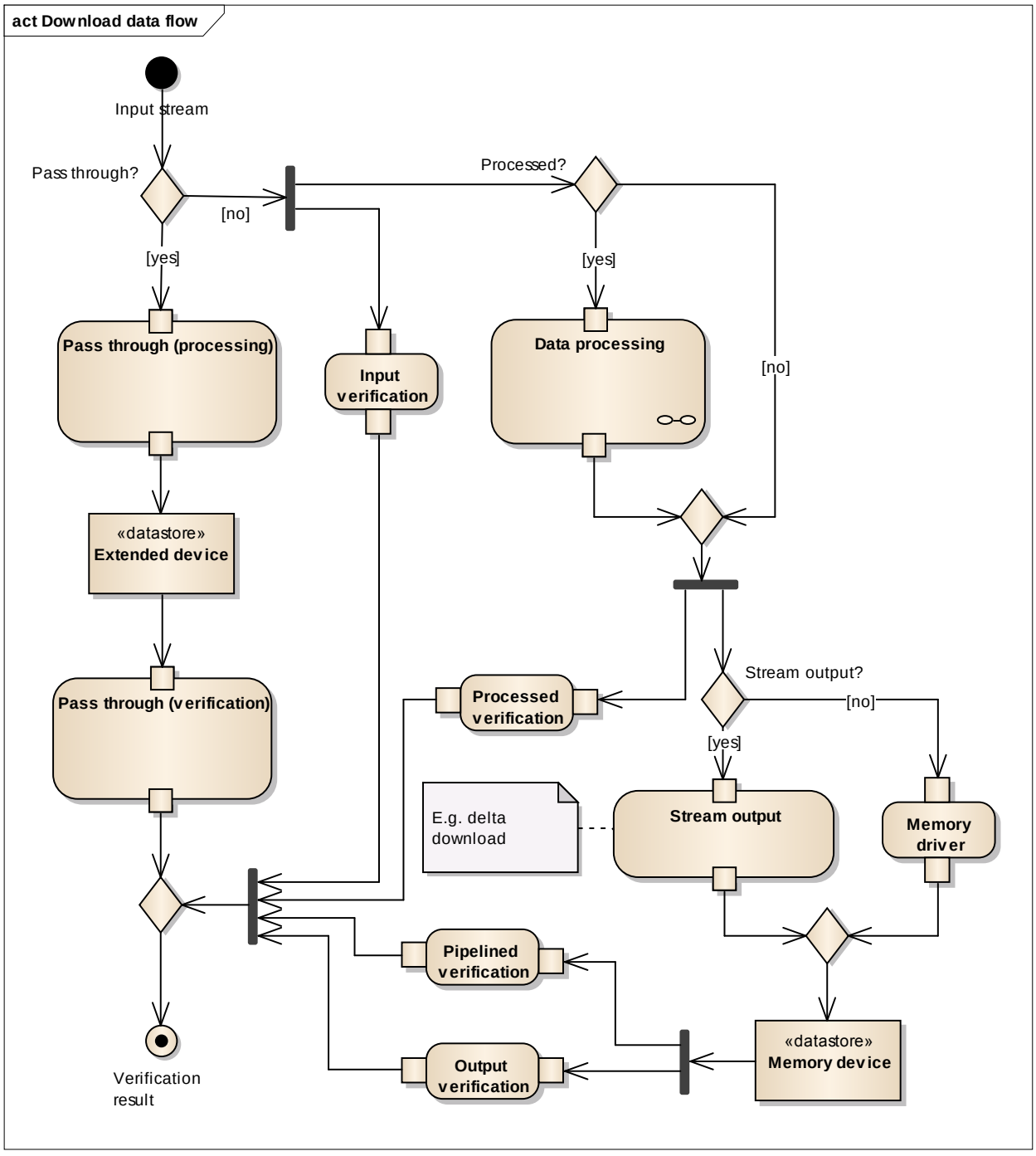


Figure 3-1 Data Flow

3.1 Features

The basic feature of this component is to encapsulate requests to a Memory Driver. As they are:

- Erase and
- Write

of flash memory. Further the

- Verification,

using the Security Module, is offered as a service to the upper layer. In addition, these requests can be redirected for

- Gap Fill,
- Pipelined Programming,
- Pipelined Verification,
- Data Processing,
- Stream Output or
- Passthrough.



Note
Not all features are available for each platform, OEM and may have to be purchased separately.

3.1.1 Gap Fill

If Gap Fill is enabled, unused parts of the written flash memory are filled with a default pattern.

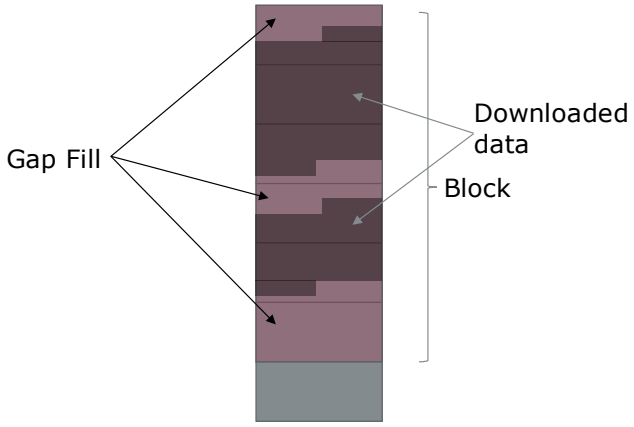


Figure 3-2 Gap Fill

If Pipelined Programming is enabled, the filling happens “pipelined” during idle times in the download sequence and after all payload data is written.

3.1.2 Pipelined Programming

Pipelined Programming parallelizes data transfer and write access to the memory. If the first packet of data is received, the LibMem will enqueue the data. Instead of writing it to the memory, it returns to the diagnostics which will proceed to receive the next package. During idle phases during reception of the 2nd package, the first one will be written to memory (see Figure 3-3).

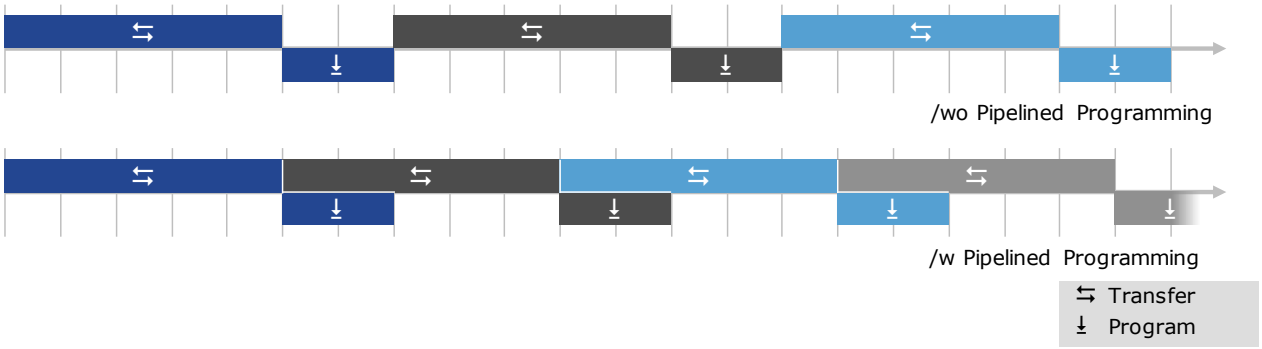


Figure 3-3 Pipelined Programming

This requires the communication stack to be linked to RAM to make it executable during memory write. If Data Processing, e. g. decompression, is enabled, it is also parallelized (see Figure 3-4).

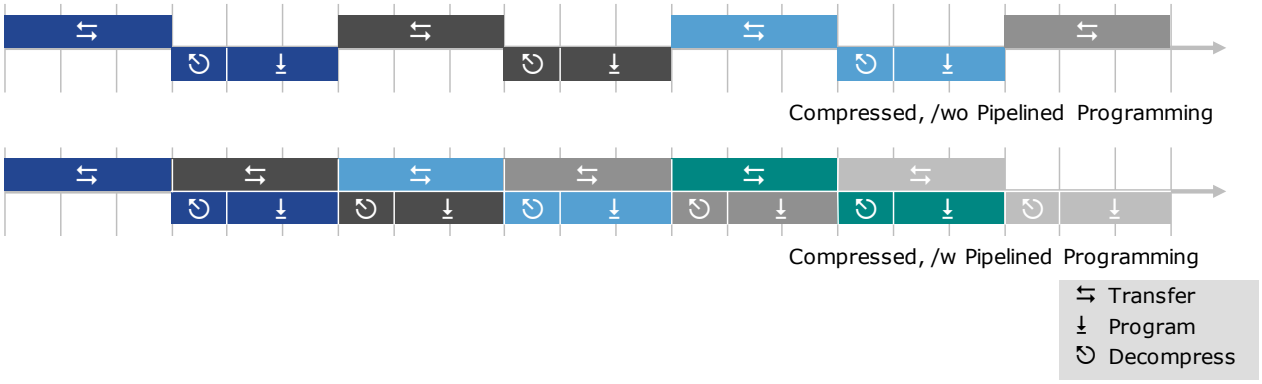


Figure 3-4 Pipelined Programming with Data Processing

This feature is especially useful if

- the time required for transferring the data is nearly the same as for processing and writing,
- the available computational power (for Data Processing) is high,
- enough RAM for the communication stack exist.

If ressources are short and/or the bus speed is high compared to the write speed, Pipelined Programming will not be useful nor possible.

Via the callout `__ApplFblMemIsPipelinedProgrammingDisabled` (see 5.9.12.1) it is possible to enable/disable pipelined Programming during runtime.

3.1.2.1 Suspend Programming

On some platforms (e.g. V850), the flash driver calls have very long latency. The resulting delays can render the use of Pipelined Programming useless. To overcome this, Suspend Programming is established.

The Flash Drivers must allow to interrupt the flash write process. If a `RxNotification()` (see 5.9.1.1) is triggered during flash write, the write process is suspended. Then, the control is returned, allowing the diagnosis to process the new data. If no further processing by the diagnosis is yet necessary, the write can be resumed.

This is done via the callouts

- `ApplFblMemDriverSuspendWrite()`
- `ApplFblMemDriverResumeWrite()`
- `ApplFblMemDriverGetWriteRemainder()`

3.1.3 Pipelined Verification

Pipelined Verification is the parallelization of data transfer and verification.

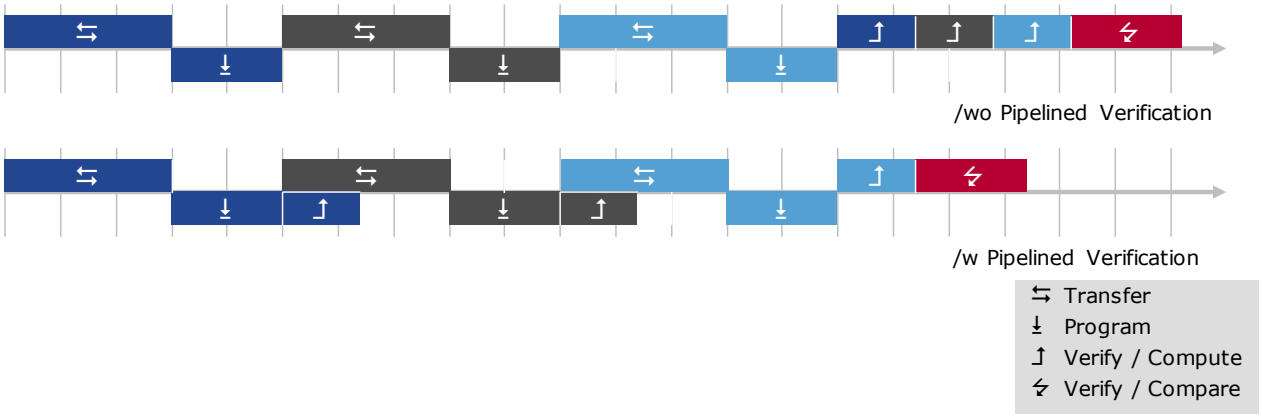


Figure 3-5 Pipelined Verification

The verification is done as long no processing of the new input data is required. Unless all data can be verified during the download, the remainder is verified after the download.

This can speed up the verification significantly. Pipelined Verification is usable in conjunction with Pipelined Programming and Data Processing.

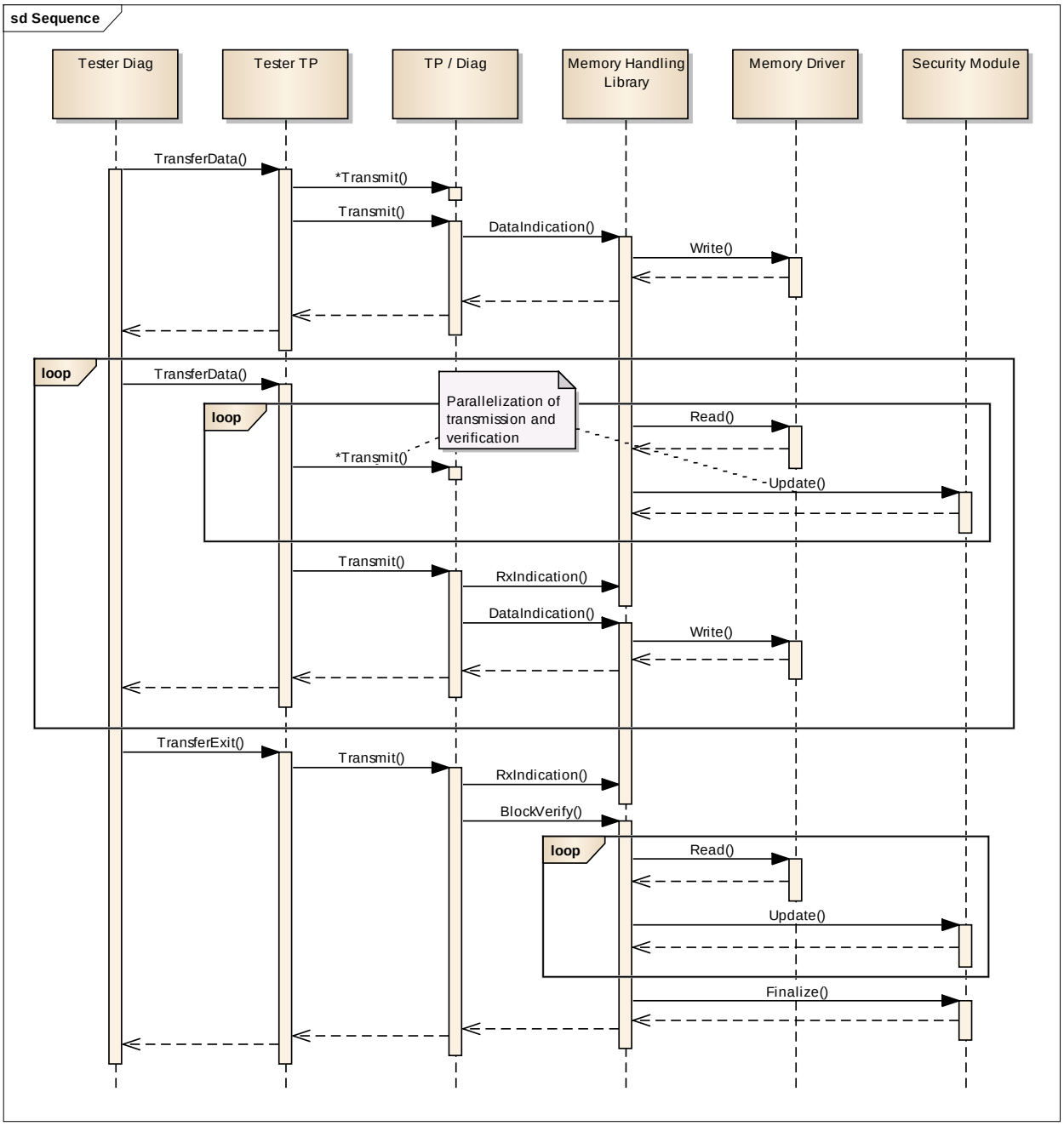


Figure 3-6 Pipelined Verification Program Flow

3.1.4 Data Processing

The write data is handed over to a decryption and / or decompression component which returns the data to write on the flash memory.

The data processing uses the callout functions

- `__ApplFblMemIsDataProcessingRequired()`
- `ApplFblInitDataProcessing()`
- `ApplFblDataProcessing()`
- `ApplFblDeinitDataProcessing()`

See 5.9.11 for details on the callouts.

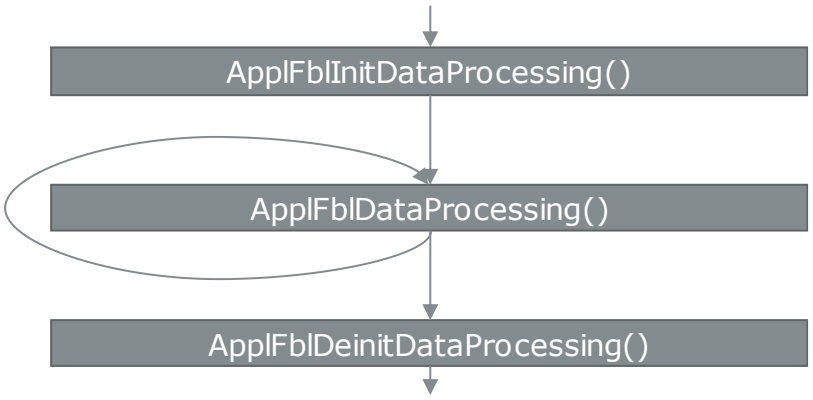


Figure 3-7 States of data processing

3.1.5 Stream Output

The write data is forwarded to another abstract flash writing entity, e. g. functionality for delta downloads. The Flash Driver is bypassed (see Figure 3-1). The write data and the affected memory region are provided to callouts.

- `__ApplFblMemIsStreamOutputRequired()`
- `ApplFblInitStreamOutput()`
- `ApplFblStreamOutput()`
- `ApplFblFinalizeStreamOutput()`
- `ApplFblDeinitStreamOutput()`

See 5.9.10 for a description of the API. The program flow for Stream Output is shown in Figure 3-8.

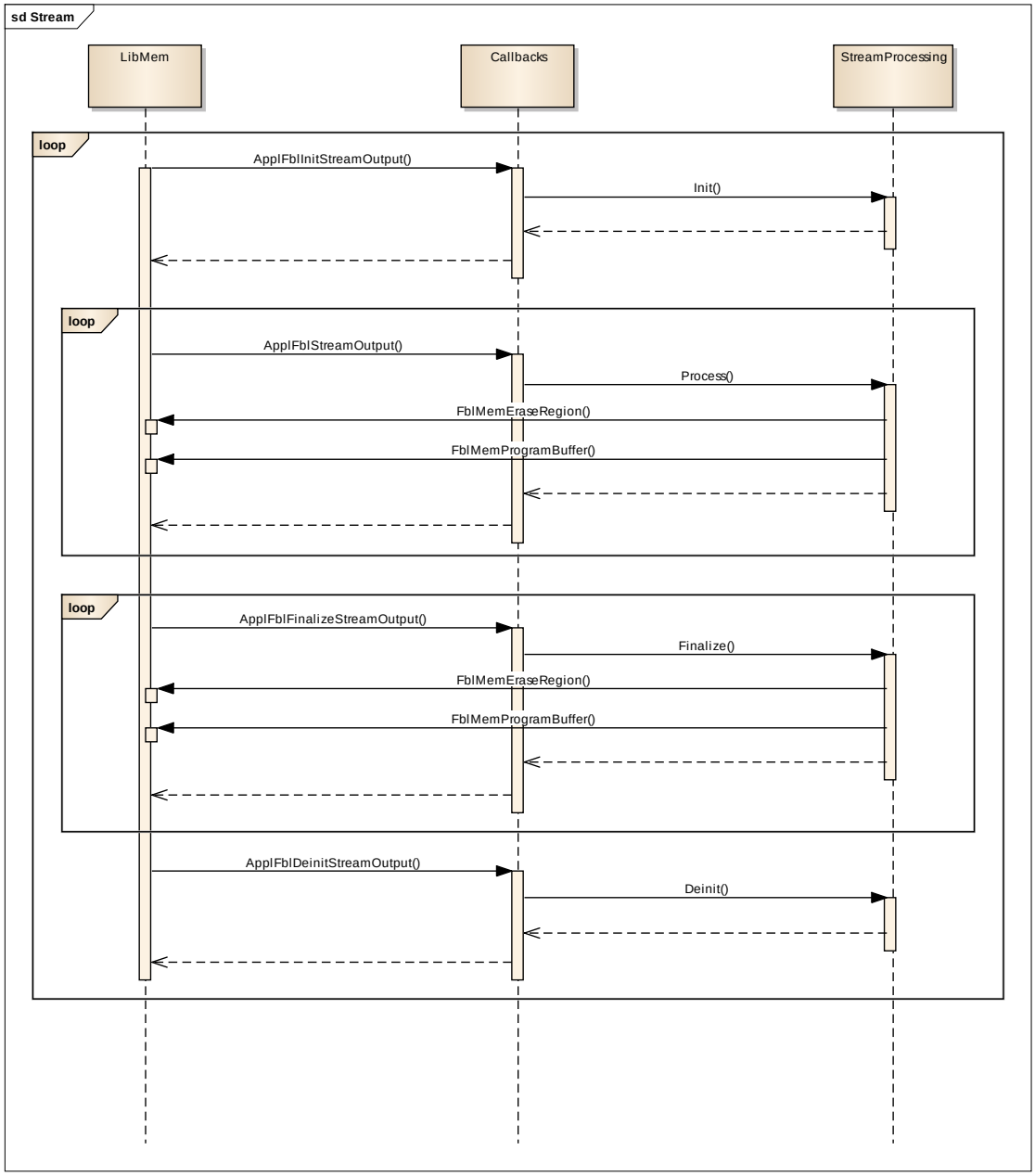


Figure 3-8 Stream Output Program Flow

3.1.6 Passthrough

This option is meant to support more advanced use-cases in a multi-processor environment where the Sub-µC should take over tasks like data processing or signature verification. The Passthrough allows the re-routing of the API calls to another instance implementing the LibMem interface. This way the Sub-µC can take over the main part of the data processing and only return the result to the Main-µC, again making the operation completely transparent for the caller (e.g. diagnostics). The re-routing is done via the callouts described in 5.9.8.

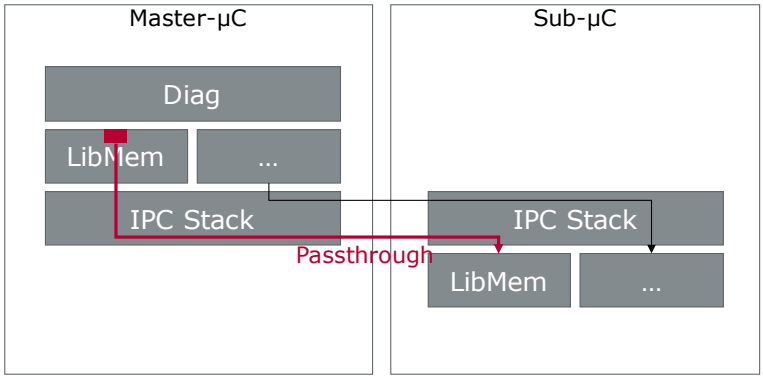


Figure 3-9 The Passthrough interface re-routes request from the upper layer (Diag) directly to a Sub-µC

If Pipelined Programming is enabled, the `DataIndication` Passthrough callout function is triggered pipelined. Instead directly forwarding the download data, it is split into smaller parts of `FBL_MEM_PASSTHROUGH_SEGMENTATION` size (see Figure 3-10). The calling of the Passthrough callouts is suspended if a `RxNotification()` arrives and continued if (new) received data is consumed by the diagnostics.

Same as for Pipelined Verification the segmentation size should be not too large (see Figure 3-16) to avoid timeouts on processing of the upper layer (the diagnostics).

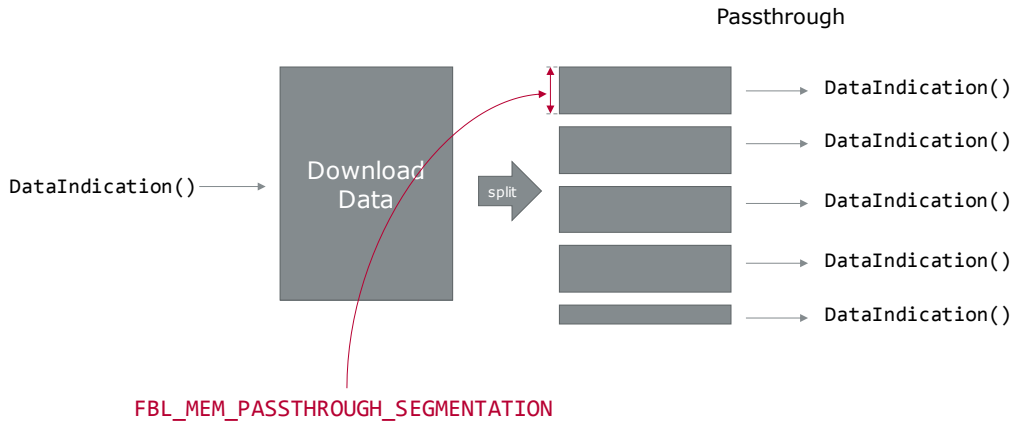


Figure 3-10 If Pipelined Programming is enabled, one DataIndication from the upper layer (diagnostics) is split into multiple pipelined Passthrough-DataIndications, each with a reduced number of data

To use the Passthrough interface

> Use the callout `__ApplFblMemIsPassThroughActive` to decide if e. g. a certain block shall be re-routed to another ECU. All calls to the LibMem referring to this block will now be re-routed via the callouts (see 5.9.8).

> Use a IPC communication stack to forward this calls to the second ECU

> Implement functionality on the second ECU to process the requests. This will include (if required) handling for compressed, encrypted data, pipelining, verification etc.

The handling of the parameters provided by the LibMem is implemented in a generic manner so that it is independent from specific OEM requirements. This approach may require pre-processing the provided information on the Sub-µC before using them. This is especially true for parameters which are only valid in the context of the Main-µC, e.g. function pointers.

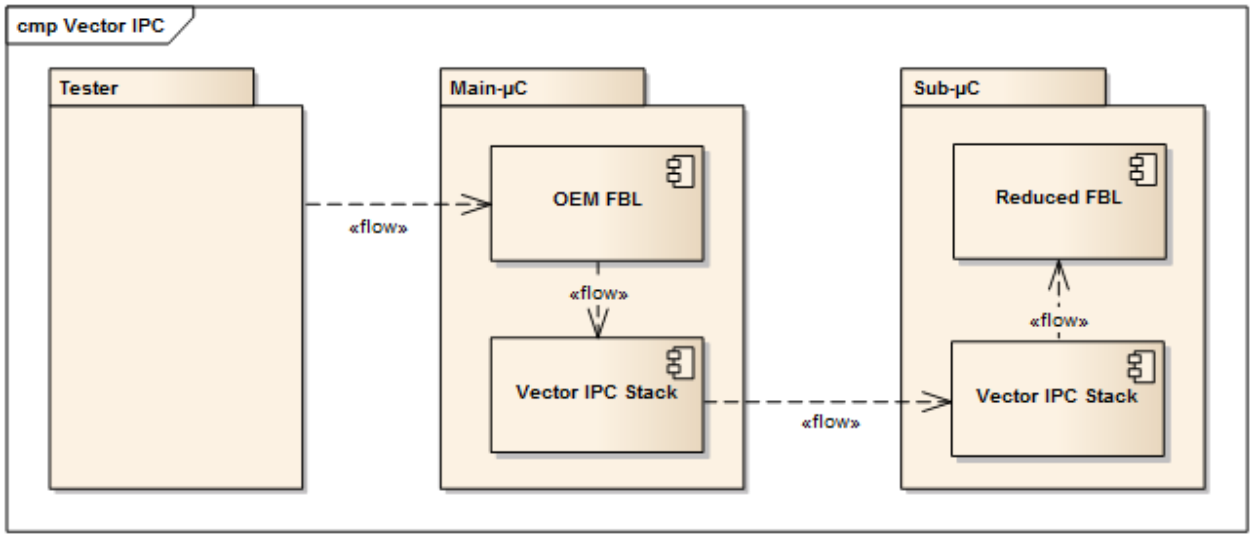


Figure 3-11 Passthrough Interface

The Passthrough interface forwards all calls to the LibMem transparently to the Sub-µC. Depending on the use case, there may be further diagnostic functionality which needs to be forwarded to the Sub-µC (Figure 3-12). These, especially OEM-specific, services are not handled by the Passthrough interface. Please see ([1]) for more information.

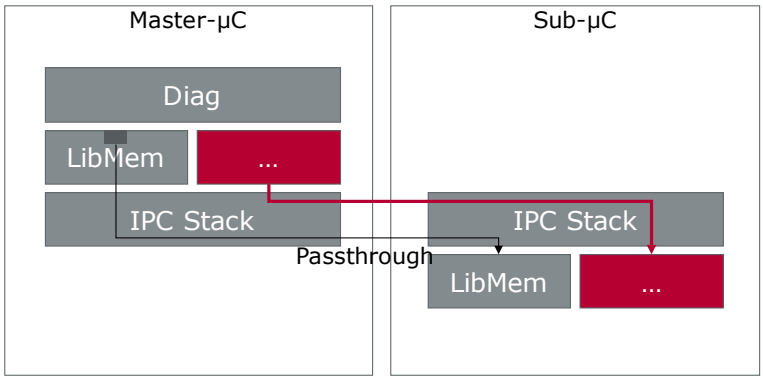


Figure 3-12 Other diagnostic services (Non-LibMem services) need to be routed separately.

3.1.7 Resumable Programming

Resumable Programming allows to continue an aborted download instead of restarting it completely. This functionality is supported (OEM specific) by the diagnostics. The download is split into resume sections. It the download is aborted, it can be continued at the last resume section.

The diagnostics manages a table of the resume sections. A resume section can consist of at least one or several complete flash blocks.

If during a download the end of a flash block is reached, the LibMem cleans up its state and notifies the diagnostics with `ApplFblMemCheckpointIndication()`. The diagnostics checks if this address corresponds with a resume sections end. If so, it asks the LibMem to create a snapshot by calling `FblMemCreateSnapshot()` from the indication callout (Figure 3-13).

Caution
`FblMemCreateSnapshot()` must be called from `ApplFblMemCheckpointIndication()` otherwise it is not guaranteed that the state of the LibMem is consistent.

The diagnostics will provide a buffer as a parameter in the call of `FblMemCreateSnapshot()`. The LibMem copies its state into this buffer and the diagnostics will store this information.

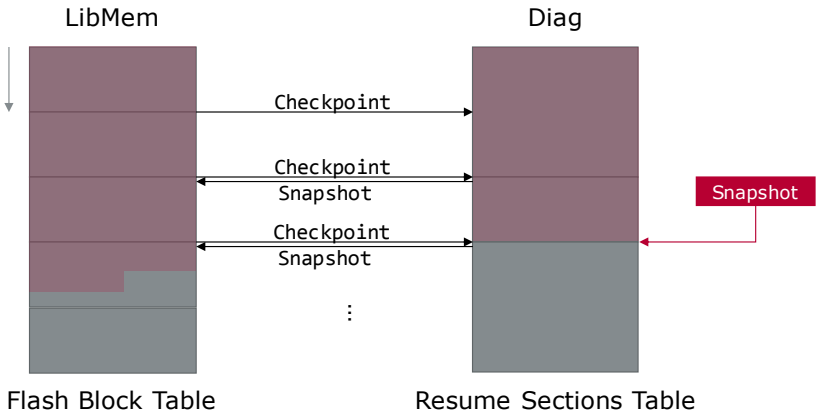


Figure 3-13 Resumable Programming

If the download is aborted after a snapshot has been stored, the diagnostics should

- Load the snapshot and check with `FblMemGetSnapshotInfo()` for its content.
- Erase the (potentially) partially written flash blocks
- Call `FblMemRestoreSnapshot()`

In Figure 3-14, the program flow for Resumable Programming is shown.

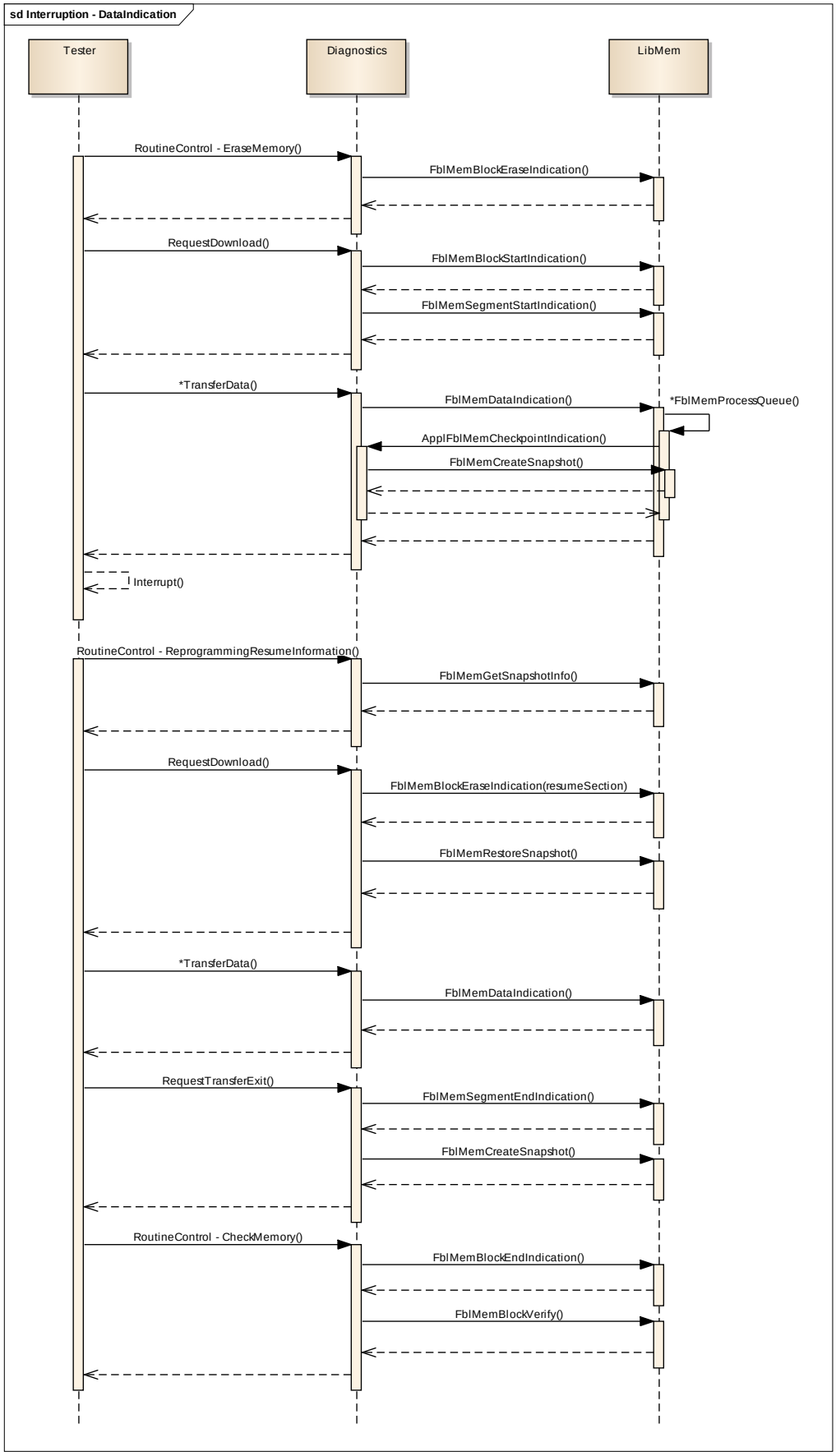


Figure 3-14 Resumable Programming Flow

3.1.8 Multiple Input Sources

The LibMem supports multiple input sources. This mainly provides different buffers for the inputs, (needed to be provided e.g. to a TP) before a download already start. To enable this,

```
#define FBL_MEM_ENABLE_MULTI_SOURCE
```

needs to be defined. In Figure 3-15 the principle is shown. The first download channel is a tester connected via e.g. UDS on CAN. A second (third, fourth …) tester is connected by another channel. This can be e.g. a IPC connection to another microcontroller (see 3.1.6) using a different diagnostic protocol (e.g. MCMP) etc.

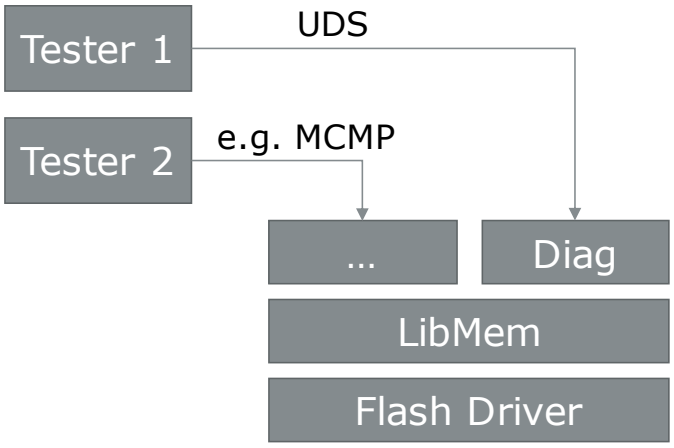


Figure 3-15 Multiple Input Sources (principle)

From the point of the LibMem the memory access to the flash of the different sources is independent from each other. The LibMem itself does not provide means to avoid collisions by accessing the same memory region by both channels at the same time. This could be addressed by e.g. the use of different blocks for each connection.

Further, the LibMem cannot address the need for synchronization between different tester connections, e.g. if one tester requests a reset but the other is still downloading.

Only one connection can be active at a time. If a block is processed completely (after verification is done), a switch to another source is possible by calling `FblMemLockInputSource()`.

3.2 Verification Variants

Verification provides measures to ensure the integrity (and authenticity) of downloaded data. This is usually done by cryptographic signature checks. The Lib Mem will pass the downloaded data to the Security Module to calculate the verification.

As shown in Figure 3-1, this can be done in different ways. Not all variants are available in each use case or for each OEM.

- **Input Verification:** the downloaded data is used for verification before eventually unpack, decrypt or write it to the flash.
- **Processed Verification:** is verification of the processed (e. g. deflated, decrypted) but not yet written data. This is a variant of Input Verification in the meaning of processing the data.

- **Output Verification:** is verification of data read from memory after writing.
- **Pipelined Verification:** is verification of data read from memory after writing (same as Output Verification), but the verification is done during idle times of the download sequence (see 3.1.3).

Generally, there are two kinds of verification functions which is reflected in the data types of the verification functions (see 5.1)

- Input Verification functions
  An Input Verification function accesses the data as a stream. It is called sequentially with a chunk of data that need to be processed.
- Output Verification functions
  The Output Verification parameter type `tFblMemVerifyParamOutput` contains a pointer to a read function. By this, the Output Verification function can acquire data according to its own needs.

Depending on OEM requirements, additional information is part of the verification data input. Like

- (starting) physical address of the data,
- logical address of the data (e. g. Logical Block, Segment etc.) or
- an initialization vector etc.

Further, depending on the used verification algorithm, credentials like certificates, keys etc. are required.

3.3 Pipelined Operations

Pipelined Programming, Pipelined Verification and GapFill will be aborted if the `RxNotification` (see 5.9.1.1) is notified. This is shown in Figure 3-16 for Pipelined Verification. The segmentation size for the operations should be selected

- as large as possible to increase data processing speed and
- as small as required to avoid P2 timeouts after a `RxNotification`.

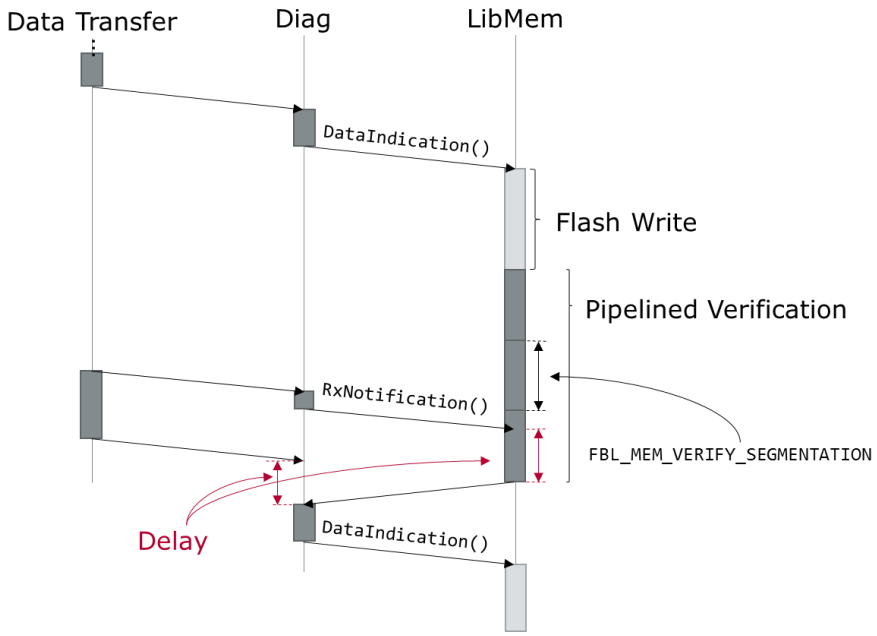


Figure 3-16 Reduced download speed on Pipelined Verification because of large VERIFY_SEGMENTATION size.

3.4 States

A typical download state flow is shown in Figure 3-17.

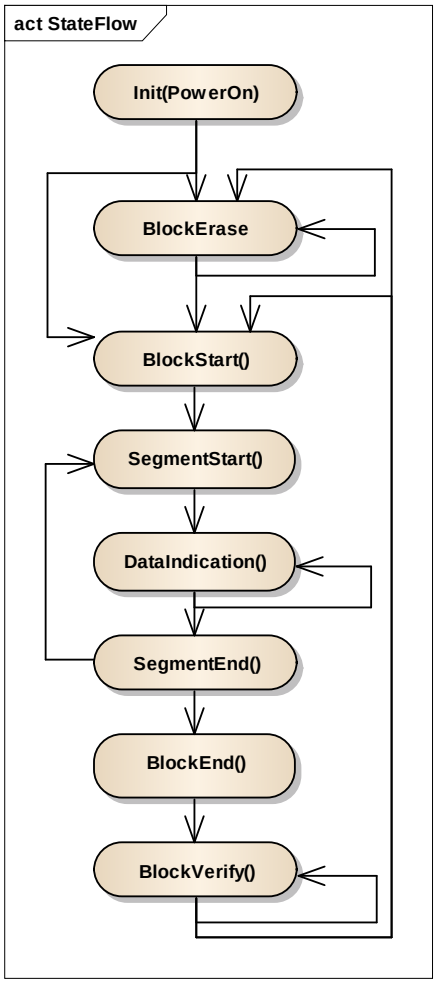


Figure 3-17 State flow of the LibMem

3.5 Main Functions

The main function is `FblMemTask()`. (see 5.2.13 Task). The LibMem uses a prioritized job queue for handling background tasks. The `FblMemTask()` will take the next job from the queue and execute it.

Depending on the configuration, different tasks can be executed directly or as a background task. E.g. a `DataIndication()` will enqueue a new job with the write data. If Pipelined Programming is disabled, all pending jobs are consumed before the `DataIndication()` returns. If Pipelined Programming is enabled, it returns without processing the jobs. The next call to `FblMemTask()` will cause the write to be executed.

If pipelining is enabled, the `FblMemTask()` processes all jobs in the queue. It returns if the queue is empty. During processing it repeatedly calls the `FblLookForWatchdog()` routine to keep the bus transmission ongoing. When a `RxNotification()` arrives, the processing is suspended. On the next call of `FblMemTask()`, the processing of jobs is continued.

If pipelining is disabled, the `FblMemTask()` does not process jobs and therefore no calls of `FblLookForWatchdog()` are issued.

3.6 Error Handling

The LibMem uses the following mechanism for error handling.

3.6.1 Development Error Reporting

Checks for configuration errors use the macro `assertFblInternal()` as it is common used throughout the FBL components.

4 Integration

This chapter gives necessary information for the integration of the FBL LibMem into a bootloader environment of an ECU.

4.1 Scope of Delivery

The delivery of the LibMem contains the files which are described in Table 4-1 and Table 4-2.

4.1.1 Static Files

File Name	Description
fbl_mem.c	This is the source file of the component.
fbl_mem.h	This is the header file of the component

Table 4-1 Static files

4.1.2 Other Files

File Name	Description
fbl_mem_oem.h	This file is provided by the Diagnostic layer (OEM specific)

Table 4-2 Other Files

5 API Description

For an interfaces overview please see Figure 2-2.

5.1 Type Definitions

The types defined by the LibMem are described in this chapter.

Type Name	C-Type	Description	Value Range
tFblMemInputSource	vuintx	Handle to an input source (see 3.1.8)	
tFblMemVerifyStatus	Configurable	Return type for Verification Function	
tFblMemVerifyFctInput	Function Pointer	Verification function	
tFblMemConstRamData	const vuint8 *		
tFblMemDfi	vuint8	Data format identifier (data processing)	

Table 5-1 Type definitions

5.1.1 tFblMemVerifyRoutineInput

This struct contains the information necessary to call an Input Verification function.

Struct Element Name	C-Type	Description	Value Range
function	tFblMemVerifyFctInput	Pointer to an input verification function (See 3.2)	
param	tFblMemVerifyParamInput *	Parameter type for verification function	

Table 5-2 tFblMemVerifyRoutineInput

5.1.2 tFblMemVerifyRoutineOutput

This struct contains the information necessary to call an Output Verification function.

Struct Element Name	C-Type	Description	Value Range
function	tFblMemVerifyFctOutput	Pointer to an input verification function (See 3.2)	

Struct Element Name	C-Type	Description	Value Range
param	tFblMemVerifyParamOutput *	Parameter type for verification function	

Table 5-3 tFblMemVerifyRoutineOutput

5.1.3 tFblMemBlockInfo

This structure contains the definition of a block.

Struct Element Name	C-Type	Description	Value Range
targetAddress	tFblAddress	Physical address where to write download data (See 5.1.7)	
targetLength	tFblLength	Physical length of write data (See 5.1.7)	
logicalAddress	tFblAddress	Logical address of download data (See 5.1.7)	
logicalLength	tFblLength	Logical length of write data (See 5.1.7)	
verifyRoutineInput	tFblMemVerifyRoutineInput	(see above)	
verifyRoutineProcessed	tFblMemVerifyRoutineInput	(see above)	
verifyRoutinePipe	tFblMemVerifyRoutineInput	(see above)	
verifyRoutineOutput	tFblMemVerifyRoutineOutput	(see above)	
readFct	tFblMemVerifyReadFct	(see below)	
segmentList	tFblMemSegmentList *	(see below)	
maxSegments	vuint8	(see below)	

Table 5-4 tFblMemBlockInfo

The `readFct` is usually redirected to a diagnostic read function, redirecting to a Flash Driver read function. This allows

- ECC-safe reading (instead an eventual ECC failure causes a trap or is corrected without notice, it is reported to the verification routine) and
- accessing non-memory mapped devices.

The `segmentList` is a list of written segments (see Figure 2-3) inside the logical block. Memory not contained in this list is unwritten. The list is empty on beginning of a download to a logical block and is filled successively during a download.

5.1.4 tFblMemVerifyData

This structure contains a buffer in RAM containing verification data (signature, checksum).

Struct Element Name	C-Type	Description	Value Range
data	tFblMemRamData	Pointer to buffer	
length	tFblLength	Length of data in buffer	

Table 5-5 tFblMemVerifyData

5.1.5 tFblMemBlockVerifyData

This structure contains the verification data (signature, checksum) for separate verification functions.

Struct Element Name	C-Type	Description	Value Range
verifyDataInput	tFblMemVerifyData		
verifyDataProcessed	tFblMemVerifyData		
verifyDataPipe	tFblMemVerifyData		
verifyDataOutput	tFblMemVerifyData		

Table 5-6 tFblMemBlockVerifyData

5.1.6 tFblMemSegmentInfo

This structure contains the information that describes one memory segment (see Figure 2-3).

Struct Element Name	C-Type	Description	Value Range
targetAddress	tFblAddress	Physical address of segment (See 5.1.7)	
targetLength	tFblLength	Physical length of segment (See 5.1.7)	
logicalAddress	tFblAddress	Logical address of segment (See 5.1.7)	
logicalLength	tFblLength	Logical address of segment (See 5.1.7)	
Type	tFblMemType	enum	kFblMemType_RAM kFblMemType_ROM
dataFormat	tFblMemDfi	Data Format Identifier (see 5.1.8)	

Table 5-7 tFblMemSegmentInfo

5.1.7 Logical and Physical Addresses

The physical address and length defines the location in the real memory. The usage of the logical address and length depends on the use case. It can be used to represent e.g. OEM

specific block indices or can be used in the `__ApplFblIs*Required()`-macros to differentiate between memory which shall be written via Pipelined Programming, Stream Output, shall be re-routed to another Sub-µC via the Passthrough interface etc.

5.1.8 Data Format Identifier

The DFI describes different formats of download data as intended by the UDS Request Download (0x34) definition. It can indicate e.g.

- standard,
- compressed or
- encrypted

data. The usage is OEM specific. Same as the logical address, this can be evaluated by the `__ApplFblIs*Required()`-macros to decide e.g. if decompression, decryption etc. is needed.

5.2 General Services provided by LibMem

5.2.1 InitPowerOn

Prototype	
`tFblMemRamData FblMemInitPowerOn( void )`	
Parameter	
–	-
Return code	
	Returns pointer on the used input buffer
Functional Description	
Initialize module	
Particularities and Limitations	
> This function is synchronous.	
Expected Caller Context	
>	

Table 5-8 InitPowerOn

5.2.2 InitPowerOnExt

Prototype	
`tFblMemRamData FblMemInitPowerOnExt(tFblLength preambleLen, tFblMemInputSource sourceHandle)`	
Parameter	
`preambleLen`	Length of preamble stored during buffer switch
`sourceHandle`	Handle of input source
Return code	
	Pointer to initial input buffer

Functional Description
Initialize module
Particularities and Limitations
> This function is synchronous.
Expected Caller Context
>

Table 5-9 InitPowerOnExt

5.2.3 Init

Prototype	
`tFblMemRamData FblMemInit ( void )`	
Parameter	
–	-
Return code	
	Pointer to active input buffer
Functional Description	
(Re)-initialize input buffers. Should be called before performing additional tasks in case a previous operation failed	
Particularities and Limitations	
> This function is synchronous. > FblMemInitPowerOn() has been executed before	
Expected Caller Context	
>	

Table 5-10 Init

5.2.4 Deinit

Prototype	
`tFblMemRamData FblMemDeinit ( void )`	
Parameter	
–	-
Return code	
`retCode`	-
Functional Description	
De-initialize module Should be called when memory driver is de-initialized	
Particularities and Limitations	
> This function is synchronous.	

Expected Caller Context
>

Table 5-11 Deinit

5.2.5 GetActiveBuffer

Prototype	
`tFblMemRamData FblMemGetActiveBuffer( void )`	
Parameter	
–	-
Return code	
	Pointer to active input buffer
Functional Description	
Return active input buffer, which can be used for current data reception Buffer pointer includes previously set offset to accommodate for alignment requirements	
Particularities and Limitations	
> This function is synchronous. > FblMemInitPowerOn() has been executed before	
Expected Caller Context	
>	

Table 5-12 GetActiveBuffer

5.2.6 BlockEraseIndication

Prototype	
`tFblMemStatus FblMemBlockEraseIndication( const V_MEMRAM1 tFblMemBlockInfo V_MEMRAM2 V_MEMRAM3 * block )`	
Parameter	
`block`	Pointer to block information structure Only address and length members have to be initialized
Return code	
`retVal`	Result of operation (potentially remapped to OEM specific NRC)
Functional Description	
Indicate start of a new block (just before first segment)	
Particularities and Limitations	
> This function is synchronous. > FblMemInitPowerOn() executed before	

Expected Caller Context
>

Table 5-13 BlockEraseIndication

5.2.7 BlockStartIndication

Prototype	
`tFblMemStatus FblMemBlockStartIndication ( const V_MEMRAM1 tFblMemBlockInfo V_MEMRAM2 V_MEMRAM3 * block )`	
Parameter	
`block`	Pointer to block information structure Required members depending on configuration
Return code	
`retVal`	Result of operation (potentially remapped to OEM specific NRC)
Functional Description	
Indicate start of a new block (just before first segment) Block may consist of one or more segments	
Particularities and Limitations	
> This function is synchronous. > FblMemInitPowerOn() has been executed before > FblMemInit() has been executed before in case previous cycle had failed	
Expected Caller Context	
>	

Table 5-14 BlockStartIndication

5.2.8 BlockEndIndication

Prototype	
`tFblMemStatus FblMemBlockEndIndication ( void )`	
Parameter	
Return code	
`retVal`	Result of operation (potentially remapped to OEM specific NRC)
Functional Description	
Indicate end of current block (after last segment) Finalize verification if configured	
Particularities and Limitations	
> This function is synchronous. > FblMemSegmentEndIndication() has been executed before	

Expected Caller Context
>

Table 5-15 BlockEndIndication

5.2.9 BlockVerify

Prototype	
`tFblMemStatus FblMemBlockVerify( const V_MEMRAM1 tFblMemBlockVerifyData V_MEMRAM2 V_MEMRAM3 * verifyData, V_MEMRAM1 tFblMemVerifyStatus V_MEMRAM2 V_MEMRAM3 * verifyResult )`	
Parameter	
`verifyData`	Pointer to verification structure Required members dependent on configuration
`verifyResult`	Pointer to extended verification result Null pointer if no extended result required
Return code	
`retVal`	Result of operation (potentially remapped to OEM specific NRC)
Functional Description	
Finalize block verification Hint: also call function if verification not configured	
Particularities and Limitations	
> This function is synchronous. > FblMemBlockEndIndication() has been executed before	
Expected Caller Context	
>	

Table 5-16 BlockVerify

5.2.10 SegmentStartIndication

Prototype
`tFblMemStatus FblMemSegmentStartIndication( const V_MEMRAM1 tFblMemSegmentInfo V_MEMRAM2 V_MEMRAM3 * segment )`

Parameter	
segment	Pointer to segment information > Target address (program operation) > Target length (program operation) > either unprocessed data length written to memory or processed data length of input > data > Logical address (verification operation) > Logical length (verification operation) > Type (RAM / ROM) > Data format identifier (data processing)
Return code	
retVal	Result of operation (potentially remapped to OEM specific NRC)
Functional Description	
Indicate start of a new segment Segment may consist of one or more chunks of input data programmed into a contiguous memory range Update verification with logical address and length if configured Initialize data processing if configured	
Particularities and Limitations	
> This function is synchronous. > FblMemBlockStartIndication() has been executed before	
Expected Caller Context	
>	

Table 5-17 SegmentStartIndication

5.2.11 SegmentEndIndication

Prototype	
`tFblMemStatus FblMemSegmentEndIndication( V_MEMRAM1 tFblLength V_MEMRAM2 V_MEMRAM3 * writeLength )`	
Parameter	
writeLength	Pointer to length written
Return code	
retVal	Result of operation (potentially remapped to OEM specific NRC) May return error generated by background task operation
Functional Description	
Indicate end of current segment Finish processing of all pending buffers, finalize data processing and program any remainder	

Particularities and Limitations
> This function is synchronous. > FblMemDataIndication() has been executed before
Expected Caller Context
>

Table 5-18 SegmentEndIndication

5.2.12 DataIndication

Prototype	
`tFblMemStatus FblMemDataIndication( tFblMemConstRamData buffer, tFblLength offset, tFblLength length )`	
Parameter	
`buffer`	Pointer to input buffer (including preamble)
`offset`	Offset of actual data (after preamble)
`length`	Length of data (without preamble)
Return code	
`retVal`	Result of operation (potentially remapped to OEM specific NRC) May return error generated by background task operation
Functional Description	
Indicate new chunk of input data > If pipelined programming is configured queue buffer and immediately return in case another input buffer is available. Otherwise finish processing of one pending input buffer. > In non-pipelined mode indicated input buffer is directly processed	
Particularities and Limitations	
> This function is synchronous. > FblMemSegmentStartIndication() has been executed before, provided buffer and data offset equal the parameters of active input buffer	
Expected Caller Context	
>	

Table 5-19 DataIndication

5.2.13 Task

Prototype	
`void FblMemTask( void )`	
Parameter	
–	-
Return code	
–	-

Functional Description
Perform background tasks If pipelined programming is configured pending buffers are processed until the operation is suspended by a Rx notification
Particularities and Limitations
> This function is synchronous. > FblMemInitPowerOn() has been executed before
Expected Caller Context
>

Table 5-20 Task

5.2.14 FlushInputData

Prototype	
`void FblMemFlushInputData ( void )`	
Parameter	
–	-
Return code	
–	-
Functional Description	
Flush any pending input data Process and program any pending input data passed via DataIndication. Pipelined verification tasks may still be pending afterwards.	
Particularities and Limitations	
> This function is synchronous. > FblMemInitPowerOn() has been executed before	
Expected Caller Context	
>	

Table 5-21 FlushInputData

5.3 Resumable Programming

5.3.1 InvalidateSnapshot

Prototype	
`void FblMemInvalidateSnapshot( tFblMemRamData snapshotBuffer, tFblLength snapshotSize )`	
Parameter	
`snapshotBuffer`	Byte buffer in which snapshot shall be stored
`snapshotSize`	Size of snapshot buffer in byte

Return code	
–	-
Functional Description	
Invalidate the contents of a snapshot buffer Invalidate both magic values and the size, so that a validity check will result in a negative result	
Particularities and Limitations	
> This function is synchronous. > FblMemInitPowerOn() has been executed before > `FBL_MEM_ENABLE_RESUMABLE_PROGRAMMING`	
Expected Caller Context	
>	

Table 5-22 InvalidateSnapshot

5.3.2 CreateSnapshot

Prototype	
`tFblMemStatus FblMemCreateSnapshot( tFblMemRamData snapshotBuffer, tFblLength snapshotSize )`	
Parameter	
`snapshotBuffer`	Byte buffer in which snapshot shall be stored
`snapshotSize`	Size of snapshot buffer in byte
Return code	
`retVal`	kFblMemStatus_Ok if snapshot creation was successful, kFblMemStatus_Failed otherwise
Functional Description	
Create a snapshot of the current programming state All relevant values required to restore the current programming state are stored in the provided buffer. This includes the information passed in block and segment indications and the current programming address. The buffer has to be large enough to hold all relevant status information, including the list of all segments. The number of the latter depends on the value passed to FblMemBlockIndication.	
Particularities and Limitations	
> This function is synchronous. > FblMemInitPowerOn has been executed before > `FBL_MEM_ENABLE_RESUMABLE_PROGRAMMING`	

Expected Caller Context
> Must be called from `ApplFblMemCheckpointIndication()`

Table 5-23 CreateSnapshot

5.3.3 RestoreSnapshot

Prototype	
`tFblMemStatus FblMemRestoreSnapshot( tFblMemRamData snapshotBuffer, V_MEMRAM1 tFblMemBlockInfo V_MEMRAM2 V_MEMRAM3 * block, V_MEMRAM1 tFblMemSegmentInfo V_MEMRAM2 V_MEMRAM3 * segment )`	
Parameter	
`snapshotBuffer`	The buffer in which the snapshot is stored
`block`	Pointer to block information structure If non-null pointer is passed information of active block at time of snapshot creation is returned
`segment`	Pointer to segment information structure If non-null pointer is passed information of active segment at time of snapshot creation is returned
Return code	
`retVal`	kFblMemStatus_Ok if snapshot was successfully restored, kFblMemStatus_Failed otherwise
Functional Description	
Restore the programming state from a snapshot All relevant values required to resume a previously saved programming state are restored. This includes the information passed in block and segment indications and the programming address	
Particularities and Limitations	
> This function is synchronous. > Snapshot created with FblMemCreateSnapshot() > FblMemInit() and FblMemBlockEraseIndication() (optional) directly called before > `FBL_MEM_ENABLE_RESUMABLE_PROGRAMMING`	
Expected Caller Context	
>	

Table 5-24 RestoreSnapshot

5.3.4 GetSnapshotInfo

Prototype	
`tFblMemStatus FblMemGetSnapshotInfo( tFblMemRamData snapshotBuffer, V_MEMRAM1 tFblAddress V_MEMRAM2 V_MEMRAM3 * validAddress )`	
Parameter	
`snapshotBuffer`	Byte buffer in which snapshot is stored

validAddress	Last valid address programmed before snapshot creation
Return code	
retVal	kFblMemStatus_Ok if snapshot information is valid, kFblMemStatus_Failed otherwise
Functional Description	
Get state of passed snapshot and last valid programmed address	
Particularities and Limitations	
> This function is synchronous. > Snapshot created with FblMemCreateSnapshot()	
Expected Caller Context	
>	

Table 5-25 GetSnapshotInfo

5.4 Low level memory operations

5.4.1 FblMemEraseRegion

Prototype	
`tFblMemStatus FblMemEraseRegion( tFblAddress eraseAddress, tFblLength eraseLength )`	
Parameter	
eraseAddress	Start address of erase region
eraseLength	Length of erase region
Return code	
retVal	Result of operation (potentially remapped to OEM specific NRC)
Functional Description	
Performs erase operation in non-volatile memory All memory segments fully or partially covered by given region are affected. Gaps in the memory segment definition are skipped.	
Particularities and Limitations	
> This function is synchronous. > Memory driver is initialized > FBL_MEM_ENABLE_ERASE	

Expected Caller Context
>

Table 5-26 FblMemEraseRegion

5.4.2 FblMemProgramBuffer

Prototype	
`tFblMemStatus FblMemProgramBuffer( tFblAddress programAddress, V_MEMRAM1 tFblLength V_MEMRAM2 V_MEMRAM3 * programLength, tFblMemRamData programData )`	
Parameter	
`programAddress`	Program address
`programLength`	Length of data (output: length programmed)
`programData`	Pointer to program data (contents are padded in case length is not aligned to memory segment size!)
Return code	
`retVal`	Result of operation (potentially remapped to OEM specific NRC)
Functional Description	
Performs program operation to non-volatile memory If the length is not aligned to the segment size the odd bytes are padded with the configured fill character. Programming may be suspended by an external event. In this case parameter programLength will be updated to reflect the length actually programmed	
Particularities and Limitations	
> This function is synchronous. > Memory driver is initialized, > address aligned to memory segment size	
Expected Caller Context	
>	

Table 5-27 FblMemProgramBuffer

5.5 Helper functions

5.5.1 FblMemSetInteger

Prototype	
`void FblMemSetInteger( vuintx count, vuint32 input, tFblMemRamData buffer )`	
Parameter	
`count`	Number of relevant bytes
`input`	Input value
`buffer`	Pointer to output buffer

Return code	
–	-
Functional Description	
Convert given integer value to big-endian byte array	
Particularities and Limitations	
>	
Expected Caller Context	
>	

Table 5-28 FblMemSetInteger

5.5.2 FblMemGetInteger

Prototype	
`vuint32 FblMemGetInteger( vuintx count, tFblMemConstRamData buffer )`	
Parameter	
`count`	Number of relevant bytes
`buffer`	Pointer to output buffer
Return code	
`Output`	Integer value
Functional Description	
Convert given big-endian byte array to integer value	
Particularities and Limitations	
>	
Expected Caller Context	
>	

Table 5-29 FblMemGetInteger

5.6 Multi Source

5.6.1 FblMemLockInputSource

Prototype	
`void FblMemLockInputSource( tFblMemInputSource sourceHandle )`	
Parameter	
`sourceHandle`	Handle of input source
Return code	
–	-
Functional Description	
Lock input to specific source	

Particularities and Limitations
> This function is synchronous.
> FBL_MEM_ENABLE_MULTI_SOURCE
Expected Caller Context
>

Table 5-30 FblMemLockInputSource

5.7 Error Handling

5.7.1 FblMemRemapStatus

Prototype	
FBL_MEM_STATUS_TYPE FblMemRemapStatus(tFblMemStatus status)	
Parameter	
status	Internal Status
Return code	
FBL_MEM_STATUS_TYPE	External status remapped to OEM specific NRC
Functional Description	
Remap error codes to OEM dependent value as defined by configuration	
Particularities and Limitations	
> This function is synchronous.	
> FBL_MEM_ENABLE_STATUS_REMAPPING	
Expected Caller Context	
>	

Table 5-31 FblMemRemapStatus

5.8 Services used by LibMem

In the following table services provided by other components, which are used by the LibMem are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
FblWd	FblLookForWatchdog()
SecMod	SecM_Verification()
SecMod	SecM_VerifySignature()
MemDriver	MemDriver_RWriteSync()
MemDriver	MemDriver_REraseSync()
MemDriver	MemDriver_RReadSync()
Diag	DiagExRCRResponsePending()
Diag	FblRealTimeSupport()

Component	API
…	

Table 5-32 Services used by the LibMem

5.9 Configurable Interfaces

5.9.1 Notifications

At its configurable interfaces the LibMem defines notification functions that are be called by other modules. The mapping is not statically defined by the LibMem but can be performed at configuration time. The function prototypes that can be used for the configuration must match the appropriate function prototype signatures, which are described in the following sub-chapters.

5.9.1.1 RxNotification

Prototype	
`void FblMemRxNotification( void )`	
Parameter	
–	-
Return code	
–	-
Functional Description	
Suspend any pending programming operation If memory driver supports pausing of active operation call respective user callback too	
Particularities and Limitations	
> This function is synchronous. > `FblMemInitPowerOn()` executed before > Enabled by `FBL_MEM_ENABLE_PIPELINING`	
Expected Caller Context	
>	

Table 5-33 RxNotification

5.9.1.2 FblMemResumeIndication

Prototype	
`void FblMemResumeIndication ( void )`	
Parameter	
–	-
Return code	
–	-

Functional Description
Resume suspended operation If pipelined programming is configured a pending operation which was suspended earlier through a RX notification will be resumed
Particularities and Limitations
> This function is synchronous. > `FblMemInitPowerOn()` executed before > Enabled by `FBL_MEM_ENABLE_PIPELINING`
Expected Caller Context
>

Table 5-34 FblMemResumeIndication

5.9.2 Callout Functions

At its configurable interfaces the LibMem defines callout functions as function-like macros. It is the integrator's task to provide the corresponding definitions. The definitions of the callouts can be adjusted to the system's needs. The LibMem callout function declarations are described in the following tables.

The most callouts will be usually configured automatically by the OEM specific header file `fbl_mem_oem.h`.

5.9.3 General Callouts

The following general callout functions are used by the LibMem

5.9.3.1 __ApplFblMemCopyBuffer

Prototype	
`__ApplFblMemCopyBuffer(address, data, length)`	
Parameter	
`address`	Target address
`data`	Source address
`length`	Length of data to write to target address
Return code	
-	-
Functional Description	
Abstraction of `memcpy()`.	
Particularities and Limitations	
>	
Call context	
>	

Table 5-35 __ApplFblMemCopyBuffer

5.9.3.2 __ApplFblMemEnterCriticalSection

Prototype	
`void __ApplFblMemEnterCriticalSection(void)`	
Parameter	
-	-
Return code	
-	-
Functional Description	
E.g. disable interrupts before entering a critical section	
Particularities and Limitations	
>	
Call context	
>	

Table 5-36 __ApplFblMemEnterCriticalSection

5.9.3.3 __ApplFblMemLeaveCriticalSection

Prototype	
`void __ApplFblMemLeaveCriticalSection(void)`	
Parameter	
-	-
Return code	
-	-
Functional Description	
E.g. re-enable interrupts after leaving a critical section	
Particularities and Limitations	
>	
Call context	
>	

Table 5-37 __ApplFblMemLeaveCriticalSection

5.9.3.4 __ApplFblMemReportProgress

Prototype	
`void __ApplFblMemReportProgress (tFblMemProgressInfo * pProgressInfo)`	
Parameter	
`pProgressInfo`	Pointer to a `tFblMemProgressInfo` structure for output.

Return code	
-	-
Functional Description	
Reports progress of current operation	
Particularities and Limitations	
>	
Call context	
>	

Table 5-38 __ApplFblMemReportProgress

5.9.4 Memory access Callouts

The following callout functions can be used to react on certain memory access during runtime.

5.9.4.1 __ApplFblMemDriverReady

Prototype	
`tFblMemStatus __ApplFblMemDriverReady(memSegment)`	
Parameter	
`memSegment`	Affected memory segment
Return code	
kFblMemStatus_Ok	specific memory driver is ready
else	specific memory driver is not ready
Functional Description	
Check whether specific memory driver is ready	
Particularities and Limitations	
>	
Call context	
>	

Table 5-39 __ApplFblMemDriverReady()

5.9.4.2 __ApplFblMemConditionCheck

Prototype	
`tFblMemStatus __ApplFblMemConditionCheck()`	
Parameter	
-	-
Return code	
kFblMemStatus_Ok	when conditions are fulfilled
else	or set custom return code when not

Functional Description
Check conditions just before and after executing memory operations E.g. check for programming voltage and set NRC 0x92 (kDiagNrcVoltageTooHigh) or 0x93 (kDiagNrcVoltageTooLow)
Particularities and Limitations
>
Call context
>

Table 5-40 __ApplFblMemConditionCheck

5.9.4.3 __ApplFblMemPreWrite

Prototype	
`void __ApplFblMemPreWrite(void)`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Perform actions directly before memory driver write	
Particularities and Limitations	
>	
Call context	
>	

Table 5-41 __ApplFblMemPreWrite()

5.9.4.4 __ApplFblMemPostWrite

Prototype	
`void __ApplFblMemPostWrite(void)`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Perform actions directly after memory driver write	
Particularities and Limitations	
>	

Call context
>

Table 5-42 __ApplFblMemPostWrite

5.9.4.5 __ApplFblMemPreErase

Prototype	
`void __ApplFblMemPreErase(void)`	
Parameter	
–	-
Return code	
–	-
Functional Description	
Perform actions directly before memory driver erase	
Particularities and Limitations	
>	
Call context	
>	

Table 5-43 __ApplFblMemPreErase

5.9.4.6 __ApplFblMemPostErase

Prototype	
`void __ApplFblMemPostErase(void)`	
Parameter	
–	-
Return code	
–	-
Functional Description	
Perform actions directly after memory driver erase	
Particularities and Limitations	
>	
Call context	
>	

Table 5-44 __ApplFblMemPostErase

5.9.4.7 __ApplFblMemPostSegmentEnd

Prototype	
`void __ApplFblMemPostSegmentEnd(void)`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Perform actions directly after segment end indication	
Particularities and Limitations	
>	
Call context	
>	

Table 5-45 __ApplFblMemPostSegmentEnd

5.9.5 Watchdog and Response Pending Callouts

Timing-related callouts like Watchdog or Response-Pending handling can be adapted by the following callouts.

5.9.5.1 __ApplFblMemWdTrigger

Prototype	
`__ApplFblMemWdTrigger(void)`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Default watchdog trigger. This is mapped to `FblLookForWatchdog()` as default. By defining `FBL_MEM_TRIGGER_STATUS_OVERWRITE`, it is possible to overwrite this setting. In this case, `FBL_MEM_WD_TRIGGER_DEFAULT` may need to be adapted.	
Particularities and Limitations	
>	
Call context	
>	

Table 5-46 __ApplFblMemWdTrigger

5.9.5.2 __ApplFblMemAdaptiveRcrRp

Prototype	
`void __ApplFblMemAdaptiveRcrRp(void)`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Watchdog trigger including adaptive generation of RCR-RP	
Particularities and Limitations	
>	
Call context	
>	

Table 5-47 __ApplFblMemAdaptiveRcrRp

5.9.5.3 __ApplFblMemForcedRcrRp

Prototype	
`void __ApplFblMemForcedRcrRp(void)`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Forced generation of RCR-RP (hint: use kForceSendRpIfNotInProgress if available)	
>	
Call context	
>	

Table 5-48 __ApplFblMemForcedRcrRp

5.9.5.4 __ApplFblMemIsRcrRpActive

Prototype	
`void __ApplFblMemIsRcrRpActive(void)`	
Parameter	
-	-

Return code	
–	-
Functional Description	
Check whether RCR-RP is already active	
Particularities and Limitations	
>	
Call context	
>	

Table 5-49 __ApplFblMemIsRcrRpActive

5.9.6 Resumable Programming Callouts

Resumable Programming uses the following callout function.

5.9.6.1 ApplFblMemCheckpointIndication

Prototype	
`void ApplFblMemCheckpointIndication( tFblAddress validAddress )`	
Parameter	
`validAddress`	Last successfully programmed address (last address of flash block)
Return code	
–	-
Functional Description	
Callout function used to indicate checkpoint for possible snapshot creation Is executed on every crossing of a flash block boundary. When a snapshot of the current programming state should be created the callout must use the API function FblMemCreateSnapshot, providing a sufficiently large buffer to hold the snapshot. In case FblMemCreateSnapshot reports an error, LibMem will continue the programming operation, but the caller may have to remember the error condition to initiate required measures (e.g. report error, abort programming, invalidate internal snapshot state).	
Particularities and Limitations	
>	
Expected Caller Context	
>	

Table 5-50 ApplFblMemCheckpointIndication

5.9.7 Suspend Programming Callouts

Suspended Programming needs the following callouts to be implemented

5.9.7.1 ApplFblMemDriverSuspendWrite

Prototype
`tFblResult ApplFblMemDriverSuspendWrite()`

Parameter	
-	-
Return code	
kFblOk	Flash driver write is suspended
kFblFailed	Flash driver write could not be suspended
Functional Description	
Suspend flash driver processing	
Particularities and Limitations	
>	
Call context	
>	

Table 5-51 ApplFblMemDriverSuspendWrite

5.9.7.2 ApplFblMemDriverResumeWrite

Prototype	
tFblResult ApplFblMemDriverResumeWrite()	
Parameter	
kFblOk	Flash driver write is resumed
kFblFailed	Flash driver write is not resumed
Return code	
-	-
Functional Description	
Resume flash driver processing	
Particularities and Limitations	
>	
Call context	
>	

Table 5-52 ApplFblMemDriverResumeWrite

5.9.7.3 ApplFblMemDriverGetWriteRemainder

Prototype	
tFblResult ApplFblMemDriverGetWriteRemainder(tFblLength remainingLength)	
Parameter	
remainingLength	
Return code	
kFblOk	

kFblFailed	
Functional Description	
Get information about unprocessed buffer contents	
Particularities and Limitations	
>	
Call context	
>	

Table 5-53 ApplFblMemDriverGetWriteRemainder

5.9.8 Passthrough Callouts

The following callout functions implement the Passthrough interface.

5.9.8.1 __ApplFblMemIsPassThroughActive

Prototype	
`tFblResult __ApplFblMemIsPassThroughActive(tFblMemBlockInfo blockInfo)`	
Parameter	
blockInfo	Current block
Return code	
kFblOk	Passthrough is active for this block. (Remote processing)
kFblFailed	Passthrough is active for this block. (Local processing)
Functional Description	
Decide if Passthrough is active for this block.	
Particularities and Limitations	
>	
Call context	
>	

Table 5-54 __ApplFblMemIsPassThroughActive

5.9.8.2 __ApplFblMemDataIndication

Prototype	
`tFblMemStatus __ApplFblMemDataIndication(tFblMemRamData activeBuffer, tFblLength actualLength)`	
Parameter	
activeBuffer	
actualLength	
Return code	
kFblMemStatus_Ok	-
else	

Functional Description
Pass request to configured routine, error code reflects result of operation
Particularities and Limitations
>
Call context
>

Table 5-55 __ApplFblMemDataIndication

5.9.8.3 __ApplFblMemBlockEraseIndication

Prototype	
`tFblMemStatus __ApplFblMemBlockEraseIndication(const V_MEMRAM1 tFblMemBlockInfo V_MEMRAM2 V_MEMRAM3 * pBlock)`	
Parameter	
`pBlock`	Pointer to block information structure Only address and length members must be initialized
Return code	
`kFblMemStatus_Ok`	Result of operation (potentially remapped to OEM specific NRC)
`else`	
Functional Description	
Pass request to configured routine Indicate start of a new block (just before first segment)	
Particularities and Limitations	
> FblMemInitPowerOn executed before	
Call context	
>	

Table 5-56 __ApplFblMemBlockEraseIndication

5.9.8.4 __ApplFblMemBlockStartIndication

Prototype	
`tFblMemStatus __ApplFblMemBlockStartIndication(V_MEMRAM1 tFblMemBlockInfo V_MEMRAM2 V_MEMRAM3 * pBlock)`	
Parameter	
`pBlock`	Pointer to block information structure Required members depending on configuration
Return code	
`kFblMemStatus_Ok`	Result of operation (potentially remapped to OEM specific NRC)
`else`	

Functional Description
Pass request to configured routine Indicate start of a new block (just before first segment) Block may consist of one or more segments
Particularities and Limitations
>
Call context
>

Table 5-57 __ApplFblMemBlockStartIndication

5.9.8.5 __ApplFblMemBlockEndIndication

Prototype	
`tFblMemStatus __ApplFblMemBlockEndIndication (void)`	
Parameter	
-	-
Return code	
`kFblMemStatus_Ok`	Result of operation (potentially remapped to OEM specific NRC)
`else`	
Functional Description	
Pass request to configured routine Indicate end of current block (after last segment) Finalize verification if configured	
Particularities and Limitations	
>	
Call context	
>	

Table 5-58 __ApplFblMemBlockEndIndication

5.9.8.6 __ApplFblMemSegmentStartIndication

Prototype
`tFblMemStatus __ApplFblMemSegmentStartIndication(const V_MEMRAM1 tFblMemSegmentInfo V_MEMRAM2 V_MEMRAM3 * pSegment)`

Parameter	
pSegment	Pointer to segment information > Target address (program operation) > Target length (program operation) > either unprocessed data length written to memory or processed data length of input > data > Logical address (verification operation) > Logical length (verification operation) > Type (RAM / ROM) Data format identifier (data processing)
Return code	
kFblMemStatus_Ok	Result of operation (potentially remapped to OEM specific NRC)
else	
Functional Description	
Pass request to configured routine Indicate start of a new segment Segment may consist of one or more chunks of input data programmed into a contiguous memory range Update verification with logical address and length if configured Initialize data processing if configured	
Particularities and Limitations	
>	
Call context	
>	

Table 5-59 __ApplFblMemSegmentStartIndication

5.9.8.7 __ApplFblMemSegmentEndIndication

Prototype	
tFblMemStatus __ApplFblMemSegmentEndIndication(V_MEMRAM1 tFblLength V_MEMRAM2 V_MEMRAM3 * writeLength)	
Parameter	
writeLength	Pointer to length written
Return code	
kFblMemStatus_Ok	Result of operation (potentially remapped to OEM specific NRC) May return error generated by background task operation
else	

Functional Description
Pass request to configured routine Indicate end of current segment Finish processing of all pending buffers, finalize data processing and program any remainder
Particularities and Limitations
>
Call context
>

Table 5-60 __ApplFblMemSegmentEndIndication

5.9.8.8 __ApplFblMemBlockVerify

Prototype	
`tFblMemStatus __ApplFblMemBlockVerify(const V_MEMRAM1 tFblMemBlockVerifyData V_MEMRAM2 V_MEMRAM3 * verifyData, V_MEMRAM1 tFblMemVerifyStatus V_MEMRAM2 V_MEMRAM3 * verifyResult, V_MEMRAM1 tFblMemBlockInfo V_MEMRAM2 V_MEMRAM3 * pBlock)`	
Parameter	
`verifyData`	Pointer to verification structure Required members dependent on configuration
`verifyResult`	Pointer to extended verification result Null pointer if no extended result required
`pBlock`	
Return code	
`kFblMemStatus_Ok`	Result of operation (potentially remapped to OEM specific NRC)
`else`	
Functional Description	
Pass request to configured routine Finalize block verification Hint: also called function if verification not configured Non-symmetric interface used here to allow passing of additional information (e.g. CRC total)	
Particularities and Limitations	
>	
Call context	
>	

Table 5-61 __ApplFblMemBlockVerify

5.9.9 Verification Callouts

The mode of verification can be altered during runtime using this callout functions.

5.9.9.1 __ApplFblMemVerifyInputIsAddress…

Prototype	
`tFblResult __ApplFblMemVerifyInputIsAddressAndLengthIncluded(V_MEMRAM1 tFblMemBlockInfo V_MEMRAM2 V_MEMRAM3 * pBlockInfo, tFblMemSegmentInfo * pSegmentInfo)`	
Parameter	
`pBlockInfo`	
`pSegmentInfo`	
Return code	
`kFblOk`	-
`kFblFailed`	
Functional Description	
Run-time decision whether address and length info is included in verification on input data	
Particularities and Limitations	
>	
Call context	
>	

Table 5-62 __ApplFblMemVerifyInputIsAddressAndLengthIncluded

5.9.9.2 __ApplFblMemVerifyProcessedIsAddress…

Prototype	
`tFblResult __ApplFblMemVerifyProcessedIsAddressAndLengthIncluded(V_MEMRAM1 tFblMemBlockInfo V_MEMRAM2 V_MEMRAM3 * pBlockInfo, tFblMemSegmentInfo * pSegmentInfo)`	
Parameter	
`pBlockInfo`	
`pSegmentInfo`	
Return code	
`kFblOk`	-
`kFblFailed`	
Functional Description	
Run-time decision whether address and length info is included in verification on processed data	
Particularities and Limitations	
>	
Call context	
>	

Table 5-63 __ApplFblMemVerifyProcessedIsAddressAndLengthIncluded

5.9.9.3 __ApplFblMemVerifyPipelinedIsAddress…

Prototype	
`tFblResult __ApplFblMemVerifyPipelinedIsAddressAndLengthIncluded(static V_MEMRAM1 tFblMemBlockInfo V_MEMRAM2 V_MEMRAM3 * pBlockInfo, tFblMemSegmentInfo * pSegmentInfo)`	
Parameter	
`pBlockInfo`	
`pSegmentInfo`	
Return code	
`kFblOk`	-
`kFblFailed`	
Functional Description	
Run-time decision whether address and length info is included in pipelined verification	
Particularities and Limitations	
>	
Call context	
>	

Table 5-64 __ApplFblMemVerifyPipelinedIsAddressAndLengthIncluded

5.9.10 Stream Output Callouts

The Stream Output interface is implemented by the following callout functions.

5.9.10.1 __ApplFblMemIsStreamOutputRequired

Prototype	
`tFblResult ApplFblMemIsStreamOutputRequired(tFblMemDfi dataFormat)`	
Parameter	
`dataFormat`	Data format identifier (data processing)
Return code	
`kFblOk`	-
`kFblFailed`	
Functional Description	
Check whether specific data format identifier implies the need for stream output	
Particularities and Limitations	
>	
Call context	
>	

Table 5-65 ApplFblMemIsStreamOutputRequired

5.9.10.2 ApplFblInitStreamOutput

Prototype	
`tFblResult ApplFblInitStreamOutput(V_MEMRAM1 tFblMemStreamProcessing V_MEMRAM2 V_MEMRAM3 * streamParam)`	
Parameter	
`streamParam`	
`->baseAddress`	Base address of altered memory region
`->baseLength`	Length of altered memory region
`->address`	Unused
`->length`	Unused
`->inputData`	Unused
`->outputData`	Unused
`->watchdog`	Watchdog Trigger function
`->inputLength`	Unused
`->outputSize`	Unused
`->consumedLength`	Unused
`->producedLength`	Unused
`->mode`	Data format identifier (DFI) identifies applied processing (e.g. delta download)
Return code	
`kFblOk`	if operation was successful
`kFblFailed`	otherwise
Functional Description	
This function is called to initialize the application specific stream output function.	
Particularities and Limitations	
>	
Call context	
>	

Table 5-66 ApplFblInitStreamOutput

5.9.10.3 ApplFblStreamOutput

Prototype	
`tFblResult ApplFblStreamOutput(V_MEMRAM1 tFblMemStreamProcessing V_MEMRAM2 V_MEMRAM3 * streamParam)`	
Parameter	
`streamParam`	
`->baseAddress`	Base address of altered memory region
`->baseLength`	Length of altered memory region
`->address`	Current memory address (based on previously produced data)

->length	Remaining length of memory region (based on previously produced data)
->inputData	Input data buffer
->outputData	Unused
->watchdog	Watchdog Trigger function
->inputLength	Length of provided input data
->outputSize	Unused
->consumedLength	(Output) Length of consumed input data
->producedLength	(Output) Length of produced output data / data persistently written to memory
->mode	Data format identifier (DFI) identifies applied processing (e.g. delta download)
Return code	
kFblOk	if operation was successful
kFblFailed	otherwise
Functional Description	
Stream output function	
Particularities and Limitations	
> Stream output must be initialized by call of ApplFblInitStreamOutput	
Call context	
>	

Table 5-67 ApplFblStreamOutput

5.9.10.4 ApplFblFinalizeStreamOutput

Prototype
tFblResult ApplFblFinalizeStreamOutput(V_MEMRAM1 tFblMemStreamProcessing V_MEMRAM2 V_MEMRAM3 * streamParam)
Parameter
streamParam
->baseAddress	Base address of altered memory region
->baseLength	Length of altered memory region
->address	Current memory address (based on previously produced data)
->length	Remaining length of memory region (based on previously produced data)
->inputData	Unused
->outputData	Unused
->watchdog	Watchdog Trigger function
->inputLength	Unused
->outputSize	Unused
->consumedLength	Unused
->producedLength	(Output) Length of produced output data / data persistently written to memory

->mode	Data format identifier (DFI) identifies applied processing (e.g. delta download)
Return code	
kFblOk	if operation was successful
kFblFailed	otherwise
Functional Description	
Finalize Stream output function	
Particularities and Limitations	
> Stream output must be initialized by call of ApplFblInitStreamOutput	
Call context	
>	

Table 5-68 ApplFblFinalizeStreamOutput

5.9.10.5 ApplFblDeinitStreamOutput

Prototype	
tFblResult ApplFblFinalizeStreamOutput(V_MEMRAM1 tFblMemStreamProcessing V_MEMRAM2 V_MEMRAM3 * streamParam)	
Parameter	
streamParam	
->baseAddress	Base address of altered memory region
->baseLength	Length of altered memory region
->address	Unused
->length	Unused
->inputData	Unused
->outputData	Unused
->watchdog	Watchdog Trigger function
->inputLength	Unused
->outputSize	Unused
->consumedLength	Unused
->producedLength	Unused
->mode	Data format identifier (DFI) identifies applied processing (e.g. delta download)
Return code	
kFblOk	if operation was successful
kFblFailed	otherwise
Functional Description	
De-initialize Stream output function	
Particularities and Limitations	
> Stream output must be initialized by call of ApplFblInitStreamOutput	

Call context
>

Table 5-69 ApplFblFinalizeStreamOutput

5.9.11 Data Processing Callouts

Data Processing requires the following callout functions to be implemented.

5.9.11.1 __ApplFblMemIsDataProcessingRequired

Prototype	
`tFblResult __ApplFblMemIsDataProcessingRequired(tFblMemDfi dataFormat)`	
Parameter	
`dataFormat`	Data format identifier (data processing)
Return code	
`kFblOk`	
`kFblFailed`	
Functional Description	
Check whether specific data format identifier implies the need for data processing. Is preconfigured by `fbl_mem_oem.h`.	
Particularities and Limitations	
>	
Call context	
>	

Table 5-70 __ApplFblMemIsDataProcessingRequired

5.9.11.2 ApplFblInitDataProcessing

Prototype	
`tFblResult ApplFblInitDataProcessing (V_MEMRAM1 tProcParam V_MEMRAM2 V_MEMRAM3 * procParam)`	
Parameter	
`procParam`	Pointer to processing parameter data structure.
`->wdTriggerFct`	pointer to the watchdog trigger
`->dataOutMaxLength`	maximum size of the output data
`->mode`	required mode of data processing (e.g.: decompression, decryption)
Return code	
`kFblOk`	
`kFblFailed`	

Functional Description
Prepare Data processing here.
Particularities and Limitations
>
Call context
>

Table 5-71 ApplFblInitDataProcessing

5.9.11.3 ApplFblDataProcessing

Prototype	
`tFblResult ApplFblDataProcessing (V_MEMRAM1 tProcParam V_MEMRAM2 V_MEMRAM3 * procParam)`	
Parameter	
`procParam`	Pointer to processing parameter data structure.
`->dataBuffer`	buffer of input data
`->dataLength`	length of input data
`->dataOutBuffer`	buffer for output data
`->dataOutLength`	length of data in output buffer out?
`->wdTriggerFct`	pointer to the watchdog trigger
`->dataOutMaxLength`	maximum size of the output data
`->mode`	required mode of data processing (e.g.: decompression, decryption)
Return code	
`kFblOk`	
`kFblFailed`	
Functional Description	
Process (e.g. decompress, decrypt) input data here. Return the processed data in the output buffer and adjust the `->dataOutLength` accordingly	
Particularities and Limitations	
>	
Call context	
>	

Table 5-72 ApplFblDataProcessing

5.9.11.4 ApplFblDeinitDataProcessing

Prototype
`tFblResult ApplFblDeinitDataProcessing (V_MEMRAM1 tProcParam V_MEMRAM2 V_MEMRAM3 * procParam)`

Parameter	
procParam	Pointer to processing parameter data structure
->wdTriggerFct	pointer to the watchdog trigger
->dataOutMaxLength	maximum size of the output data
->mode	required mode of data processing (e.g.: decompression, decryption)
Return code	
kFblOk	
kFblFailed	
Functional Description	
De-initialize the data processing here.	
Particularities and Limitations	
>	
Call context	
>	

Table 5-73 ApplFblDeinitDataProcessing

5.9.12 Pipelining Callouts

The behavior of Pipelined Programming can be modified on runtime using this callout function.

5.9.12.1 __ApplFblMemIsPipelinedProgramming…

Prototype	
__ApplFblMemIsPipelinedProgrammingDisabled(V_MEMRAM1 tFblMemBlockInfo V_MEMRAM2 V_MEMRAM3 * pBlockInfo, tFblMemSegmentInfo * pSegmentInfo)	
Parameter	
pBlockInfo	
pSegmentInfo	
Return code	
-	-
Functional Description	
Check whether pipelined programming must be disabled for specific block / segment / data format identifier	
Particularities and Limitations	
>	
Call context	
>	

Table 5-74 __ApplFblMemIsPipelinedProgrammingDisabled

6 Configuration

In the LibMem the attributes can be configured with the following methods:

- Configuration via Configuration Tool
- Configuration manually

The most settings are done implicitly by setting up a Bootloader configuration with the Configuration Tool. Only few special settings cannot be configured by the tool.

OEM specific configuration requirements are configured via `fbl_mem_oem.h`.

6.1 Configuration manually

Some special configuration switches are not configurable via the tool. These are:

- FBL_MEM_ENABLE_MULTI_SOURCE
  See paragraphs 3.1.8.
- FBL_MEM_ENABLE_PASSTHROUGH
  Enables the Passthrough callouts. See paragraphs 3.1.6 and 5.9.8.

© 2019 Vector Informatik GmbH

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
Logical Block	See Figure 2-3

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
FBL	Flash Boot Loader
IPC	Inter Processor Communication
FBT	Flash Block Table

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com